

Experiment-1

Aim : To implement a Regression problem using linear regression and custom ANN model for housing price prediction dataset on Kaggle.

Software used : Google Colab

Theory:

Regression is a supervised learning technique used to predict continuous values. In this experiment, we apply Linear Regression, a simple algorithm that models the relationship between independent variables and a continuous target variable by fitting a straight line. Additionally, we use a Custom Artificial Neural Network (ANN), which is a deep learning model composed of layers of interconnected neurons, capable of capturing complex, non-linear patterns in data.

The experiment uses the Housing Price Prediction dataset from Kaggle, aiming to predict house prices based on features like area, number of rooms, location, etc. The performance of the models is evaluated using metrics like Mean Squared Error (MSE) and R^2 score.

Dataset used: Housing Price Prediction dataset

Source code and Output :

```
# Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping

# Load
df = pd.read_csv('/kaggle/input/housing-price-prediction/Housing.csv')

# Label Encoding for categorical columns
categorical_cols = ['mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioning', 'prefarea', 'furnishingstatus']

le = LabelEncoder()
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

# Features and target
X = df.drop('price', axis=1)
y = df['price']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	13300000	7420	4	2	3	1	0	0	0	1	2	1	0
1	12250000	8960	4	4	4	1	0	0	0	1	3	0	0
2	12250000	9960	3	2	2	1	0	1	0	0	2	1	1
3	12215000	7500	4	2	2	1	0	1	0	1	3	1	0
4	11410000	7420	4	1	2	1	1	1	0	1	2	0	0

```
# Path for saving model in Kaggle
checkpoint_path = '/kaggle/working/best_ann_model.keras'

# Save best model
checkpoint = ModelCheckpoint(
    filepath=checkpoint_path,
    monitor='val_loss',
    save_best_only=True,
    mode='min',
    verbose=1
)

# Stop training early if no improvement
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10, # If val_loss doesn't improve for 10 epochs -> stop
    restore_best_weights=True,
    verbose=1
)
```

```
# Train Linear Regression
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# Predict
y_pred_lr = lin_reg.predict(X_test)

# Evaluate
mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print(f"Linear Regression - MSE: {mse_lr:.2f}, R2 Score: {r2_lr:.2f}")
```

Linear Regression - MSE: 1771751116594.04, R2 Score: 0.65

```
# Build ANN
ann = Sequential([
    Dense(128, activation='relu', input_dim=X_train.shape[1]),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])

# Compile
ann.compile(optimizer='adam', loss='mse')

# Train with callbacks
history = ann.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[checkpoint, early_stop], # Both callbacks
    verbose=1
)
```

```
# Load the best saved model
best_ann_model = tf.keras.models.load_model(checkpoint_path)

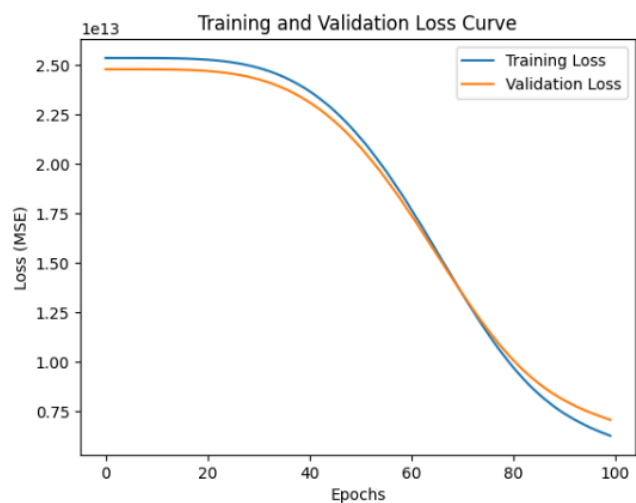
# Predict
y_pred_best = best_ann_model.predict(X_test).flatten()

# Evaluate
mse_best = mean_squared_error(y_test, y_pred_best)
r2_best = r2_score(y_test, y_pred_best)

print(f"Best Saved ANN Model - MSE: {mse_best:.2f}, R2 Score: {r2_best:.2f}")
```

4/4 ————— 0s 19ms/step
Best Saved ANN Model - MSE: 7470818212987.53, R2 Score: -0.48

```
# Plot training & validation loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE)')
plt.title('Training and Validation Loss Curve')
plt.legend()
plt.show()
```



```
# Importing necessary Libraries for visualization
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Train Linear Regression model
```

```
linear_reg = LinearRegression()
```

```
linear_reg.fit(X_train, y_train)
```

```
# Predictions using Linear Regression
```

```
y_pred_lr = linear_reg.predict(X_test)
```

```
# Predictions using Custom ANN
```

```
y_pred_ann = best_ann_model.predict(X_test).flatten() # Assuming best_ann_model is trained
```

```
# Plotting the results
```

```
plt.figure(figsize=(12, 6))
```

```
# Linear Regression Plot
```

```
plt.subplot(1, 2, 1)
```

```
plt.scatter(y_test, y_pred_lr, color='blue', alpha=0.5)
```

```
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linestyle='--')
```

```
plt.title('Linear Regression: True vs Predicted Prices')
```

```
plt.xlabel('True Prices')
```

```
plt.ylabel('Predicted Prices')
```

```
# Custom ANN Plot
```

```
plt.subplot(1, 2, 2)
```

```
plt.scatter(y_test, y_pred_ann, color='green', alpha=0.5)
```

```
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linestyle='--')
```

```
plt.title('Custom ANN: True vs Predicted Prices')
```

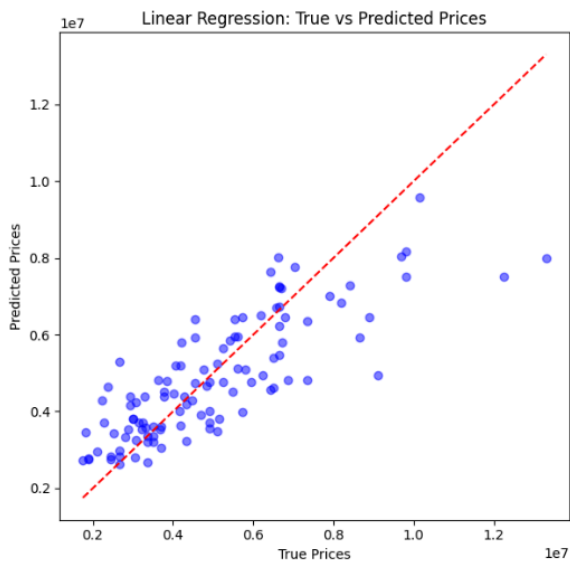
```
plt.xlabel('True Prices')
```

```
plt.ylabel('Predicted Prices')
```

```
# Show plots
```

```
plt.tight_layout()
```

```
plt.show()
```



Experiment-2

Aim : To implement Binary classification using logistic regression and custom ANN model for titanic dataset.

Software used : Google Colab

Dataset used: Titanic Dataset

Theory:

Binary classification is a supervised learning task where the goal is to classify data into one of two categories. In this experiment, we implement Logistic Regression, a statistical model that uses the logistic function to model binary outcomes. We also build a Custom Artificial Neural Network (ANN), which consists of layers of neurons that can learn complex decision boundaries. The Titanic dataset from Kaggle is used, where the objective is to predict whether a passenger survived or not based on features like age, gender, ticket class, etc. Model performance is evaluated using metrics like accuracy, precision, recall, and F1 score.

Source code :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
# Load Titanic Dataset
df = pd.read_csv('/kaggle/input/titanic-dataset/Titanic-Dataset.csv')

# View data
print(df.head())
```

```
# Drop unwanted columns
df = df.drop(['Name', 'Ticket', 'Cabin', 'PassengerId'], axis=1)

# Handle missing values
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# Encode categorical variables
le = LabelEncoder()
df['Sex'] = le.fit_transform(df['Sex']) # male -> 1, female -> 0
df['Embarked'] = le.fit_transform(df['Embarked'])

# Features and target
X = df.drop('Survived', axis=1)
y = df['Survived']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Train Logistic Regression
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

# Predict
y_pred_log = log_reg.predict(X_test)

# Evaluate
acc_log = accuracy_score(y_test, y_pred_log)
print(f'Logistic Regression Accuracy: {acc_log:.2f}')
```

Logistic Regression Accuracy: 0.80

```
# Build ANN
ann = Sequential([
    Dense(64, activation='relu', input_dim=X_train.shape[1]),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid') # Output layer for binary classification
])

# Compile
ann.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

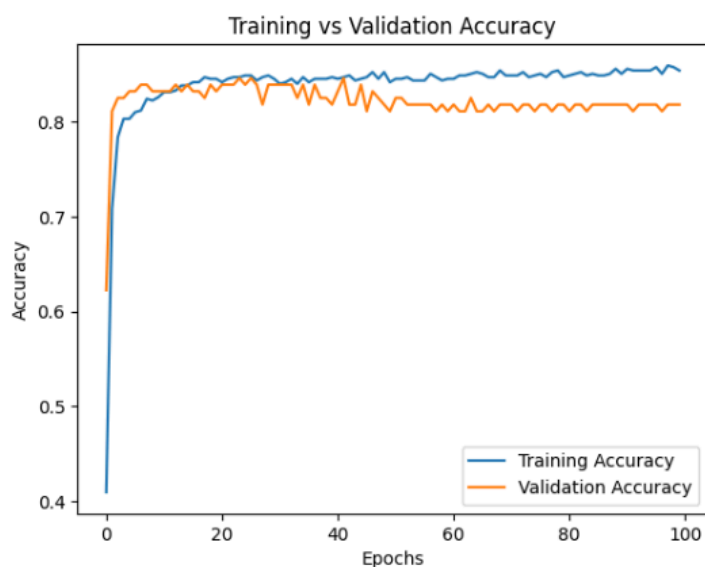
# Train
history = ann.fit(X_train, y_train, epochs=100, batch_size=32, validation_split=0.2, verbose=0)

# Predict
y_pred_ann = (ann.predict(X_test) > 0.5).astype("int32")

# Evaluate
acc_ann = accuracy_score(y_test, y_pred_ann)
print(f'Custom ANN Accuracy: {acc_ann:.2f}')
```

Output :

```
# Plot loss and accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend()
plt.show()
```



Experiment-3

Aim: To implement Multiclass classification using a custom ANN model.

Software Used: Google Colab

Dataset used: MNIST Dataset

Theory :

Multiclass classification is a supervised learning task where an input is categorized into one of three or more classes. In this experiment, a **Custom Artificial Neural Network (ANN)** is built to handle multiclass problems by using multiple output neurons and a softmax activation function. The model learns complex patterns in the data through layers of interconnected neurons and is trained using categorical cross-entropy loss. Performance is evaluated using metrics like accuracy and confusion matrix analysis.

Source Code and Output :

```
# Import Libraries
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from sklearn.preprocessing import OneHotEncoder
```

```
2025-04-28 12:57:08.391486: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1745845028.584511      31 cuda_dnn.cc:8310] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
E0000 00:00:1745845028.639753      31 cuda_blas.cc:1418] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered
```

```
# Helper Functions
def softmax(z):
    exp_z = np.exp(z - np.max(z, axis=1, keepdims=True)) # for numerical stability
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)

def cross_entropy_loss(y_true, y_pred):
    m = y_true.shape[0]
    loss = -np.sum(y_true * np.log(y_pred + 1e-9)) / m
    return loss

def accuracy(y_true, y_pred):
    return np.mean(np.argmax(y_true, axis=1) == np.argmax(y_pred, axis=1))

def relu(z):
    return np.maximum(0, z)

def relu_derivative(z):
    return (z > 0).astype(float)
```

```
# Visualization Functions
def plot_samples(X, y, n_samples=10):
    plt.figure(figsize=(15,2))
    for i in range(n_samples):
        plt.subplot(1, n_samples, i+1)
        plt.imshow(X[i].reshape(28,28), cmap='gray')
        plt.title(f"Label: {y[i]}")
        plt.axis('off')
    plt.show()

def plot_predictions(X, y_true, y_pred, n_samples=10):
    plt.figure(figsize=(15,2))
    for i in range(n_samples):
        plt.subplot(1, n_samples, i+1)
        plt.imshow(X[i].reshape(28,28), cmap='gray')
        plt.title(f"True: {y_true[i]}\nPred: {y_pred[i]}")
        plt.axis('off')
    plt.show()

# Load MNIST Data
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Flatten images
X_train = X_train.reshape(X_train.shape[0], -1) / 255.0 # (60000, 784)
X_test = X_test.reshape(X_test.shape[0], -1) / 255.0 # (10000, 784)

# One-hot encode Labels
encoder = OneHotEncoder(sparse_output=False)
y_train_oh = encoder.fit_transform(y_train.reshape(-1,1))
y_test_oh = encoder.transform(y_test.reshape(-1,1))

# Show Images Before Training
plot_samples(X_train, y_train)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 is 0us/step



```
n_hidden = 128 # hidden layer neurons

W1 = np.random.randn(n_features, n_hidden) * 0.01
b1 = np.zeros((1, n_hidden))

W2 = np.random.randn(n_hidden, n_classes) * 0.01
b2 = np.zeros((1, n_classes))

# Hyperparameters
learning_rate = 0.1
epochs = 100
batch_size = 128

# Training
for epoch in range(epochs):
    permutation = np.random.permutation(X_train.shape[0])
    X_train_shuffled = X_train[permutation]
    y_train_oh_shuffled = y_train_oh[permutation]

    for i in range(0, X_train.shape[0], batch_size):
        X_batch = X_train_shuffled[i:i+batch_size]
        y_batch = y_train_oh_shuffled[i:i+batch_size]

        # Forward pass
        Z1 = np.dot(X_batch, W1) + b1
        A1 = relu(Z1)
        Z2 = np.dot(A1, W2) + b2
        A2 = softmax(Z2)

        # Backward pass
        dZ2 = A2 - y_batch
        dW2 = np.dot(A1.T, dZ2) / X_batch.shape[0]
        dB2 = np.sum(dZ2, axis=0, keepdims=True) / X_batch.shape[0]

        dA1 = np.dot(dZ2, W2.T)
        dZ1 = dA1 * relu_derivative(Z1)
        dW1 = np.dot(X_batch.T, dZ1) / X_batch.shape[0]
        dB1 = np.sum(dZ1, axis=0, keepdims=True) / X_batch.shape[0]

        # Update
        W2 -= learning_rate * dW2
        b2 -= learning_rate * dB2
        W1 -= learning_rate * dW1
        b1 -= learning_rate * dB1

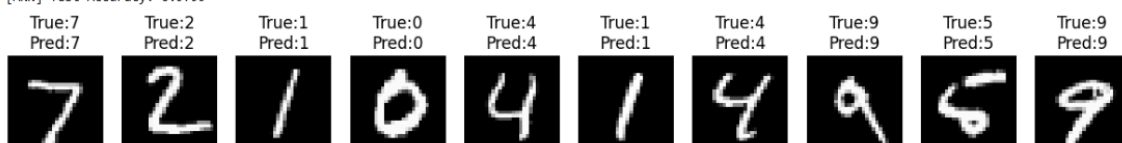
    if (epoch+1) % 10 == 0:
        Z1_train = np.dot(X_train, W1) + b1
        A1_train = relu(Z1_train)
        Z2_train = np.dot(A1_train, W2) + b2
        A2_train = softmax(Z2_train)
        train_acc = accuracy(y_train_oh, A2_train)
        print(f"[ANN] Epoch {epoch+1}, Training Accuracy: {train_acc:.4f}")

# Evaluate ANN
Z1_test = np.dot(X_test, W1) + b1
A1_test = relu(Z1_test)
Z2_test = np.dot(A1_test, W2) + b2
A2_test = softmax(Z2_test)
y_test_pred_labels_ann = np.argmax(A2_test, axis=1)

test_acc_ann = accuracy(y_test_oh, A2_test)
print(f"[ANN] Test Accuracy: {test_acc_ann:.4f}")

# Show predictions
plot_predictions(X_test, y_test, y_test_pred_labels_ann)
```

ANN Test Accuracy: 0.9799
[ANN] Test Accuracy: 0.9799



Experiment-4

Aim : To implement Train models using pretrained model and custom model: VGG16, ResNet50, MobileNet, Efficient NetB1, DenseNet121, Xception.

Software used: Jupyter Notebook

Dataset used : CIFAR dataset

Theory :

Transfer learning involves using pretrained models that have been trained on large datasets to solve new but related tasks. In this experiment, we train models using both pretrained architectures (like VGG16, ResNet50, MobileNet, EfficientNetB1, DenseNet121, and Xception) and a Custom CNN model. Pretrained models help in faster convergence and better performance by transferring learned features. Fine-tuning or feature extraction techniques are used depending on the task. Performance is evaluated using metrics like accuracy, precision, recall, and loss on the validation/test set.

Source code :

```
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, Flatten, Dropout, GlobalAveragePooling2D, Conv2D, MaxPooling2D
from tensorflow.keras.applications import VGG16, ResNet50, MobileNet, EfficientNetB1, DenseNet121, Xception
from tensorflow.keras.applications.vgg16 import preprocess_input as vgg_preprocess
from tensorflow.keras.applications.resnet50 import preprocess_input as resnet_preprocess
from tensorflow.keras.applications.mobilenet import preprocess_input as mobilenet_preprocess
from tensorflow.keras.applications.efficientnet import preprocess_input as efficientnet_preprocess
from tensorflow.keras.applications.densenet import preprocess_input as densenet_preprocess
from tensorflow.keras.applications.xception import preprocess_input as xception_preprocess
from tensorflow.keras.utils import to_categorical
```

```
# 2. Load CIFAR-10 dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Normalize data
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

```
# 3. Show CIFAR10 Dataset Samples (Before Training)
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(X_train[i].astype('uint8'))
    plt.xlabel(class_names[int(y_train[i])])
plt.suptitle("Sample CIFAR-10 Images (Before Training)", fontsize=16)
plt.show()
```

/tmp/ipykernel_31/154074205.py:11: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
 plt.xlabel(class_names[int(y_train[i])])

Sample CIFAR-10 Images (Before Training)



frog



truck



truck



deer



automobile



automobile



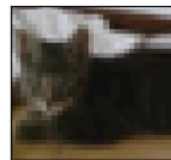
bird



horse



ship



cat

```
# 4. Define preprocessing function
def preprocess(X, model_name):
    if model_name == "VGG16":
        return vgg_preprocess(X.copy())
    elif model_name == "ResNet50":
        return resnet_preprocess(X.copy())
    elif model_name == "MobileNet":
        return mobilenet_preprocess(X.copy())
    elif model_name == "EfficientNetB1":
        return efficientnet_preprocess(X.copy())
    elif model_name == "DenseNet121":
        return densenet_preprocess(X.copy())
    elif model_name == "Xception":
        return xception_preprocess(X.copy())
    else:
        return X / 255.0

# 5. Resize data if needed (for Xception)
X_train_resize = tf.image.resize(X_train, (75,75))
X_test_resize = tf.image.resize(X_test, (75,75))

# 6. Define pretrained model builder
def build_pretrained_model(base_model_class, input_shape, preprocess_fn, name="Model", resize=False):
    if resize:
        inputs = tf.keras.Input(shape=(75,75,3))
    else:
        inputs = tf.keras.Input(shape=input_shape)

    x = preprocess_fn(inputs)

    base_model = base_model_class(include_top=False, input_tensor=x, weights='imagenet')
    base_model.trainable = False

    x = GlobalAveragePooling2D()(base_model.output)
    x = Dense(128, activation='relu')(x)
    x = Dropout(0.3)(x)
    outputs = Dense(10, activation='softmax')(x)

    model = Model(inputs, outputs, name=name)
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

```
# 7. Custom CNN model
def build_custom_cnn():
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
        MaxPooling2D(2,2),
        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D(2,2),
        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

# 8. Models to Train
models_info = {
    'CustomCNN': (build_custom_cnn, False),
    'VGG16': (lambda: build_pretrained_model(VGG16, (32,32,3), vgg_preprocess, "VGG16"), False),
    'ResNet50': (lambda: build_pretrained_model(ResNet50, (32,32,3), resnet_preprocess, "ResNet50"), False),
    'MobileNet': (lambda: build_pretrained_model(MobileNet, (32,32,3), mobilenet_preprocess, "MobileNet"), False),
    'EfficientNetB1': (lambda: build_pretrained_model(EfficientNetB1, (32,32,3), efficientnet_preprocess, "EfficientNetB1"), False),
    'DenseNet121': (lambda: build_pretrained_model(DenseNet121, (32,32,3), densenet_preprocess, "DenseNet121"), False),
    'Xception': (lambda: build_pretrained_model(Xception, (75,75,3), xception_preprocess, "Xception", resize=True), True)
}
```

```
# 9. Training + After Results
results = {}

for name, (build_fn, needs_resize) in models_info.items():
    print(f"\nTraining {name}...")

    model = build_fn()

    if needs_resize:
        history = model.fit(X_train_resize, y_train_cat, epochs=5, batch_size=64, validation_split=0.2, verbose=1)
        loss, acc = model.evaluate(X_test_resize, y_test_cat)
    else:
        history = model.fit(X_train, y_train_cat, epochs=5, batch_size=64, validation_split=0.2, verbose=1)
        loss, acc = model.evaluate(X_test, y_test_cat)

    results[name] = acc
    print(f"{name} Test Accuracy: {acc:.4f}")

# After results: predict and show
print(f"Showing predictions for {name}:")
plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)

    if needs_resize:
        img = X_test_resize[i:i+1]
    else:
        img = X_test[i:i+1]

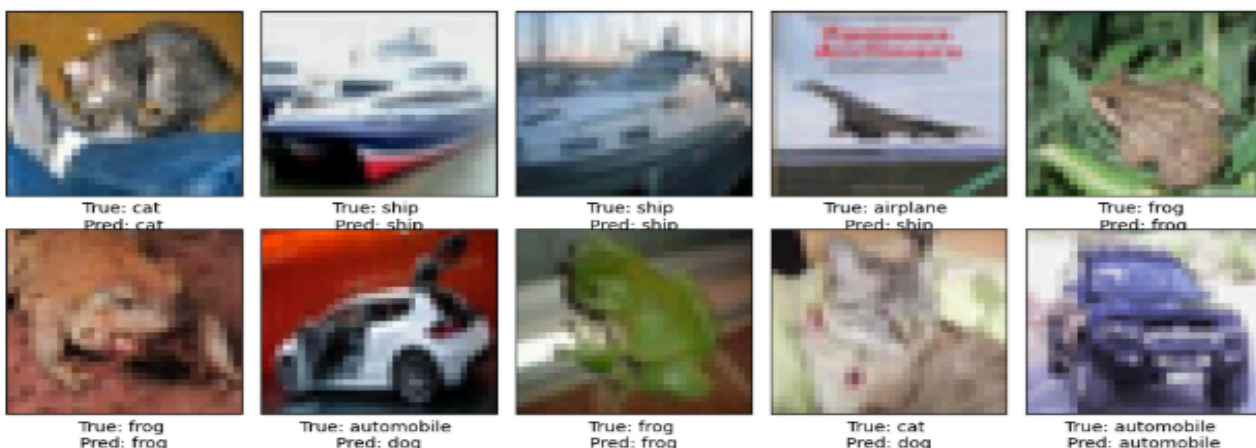
    pred = model.predict(img, verbose=0)
    pred_label = np.argmax(pred)

    plt.imshow(X_test[i].astype('uint8'))
    plt.xlabel(f"True: {class_names[int(y_test[i])]} \nPred: {class_names[pred_label]}")
plt.suptitle(f"Predictions After Training - {name}", fontsize=14)
plt.tight_layout()
plt.show()
```

Output :

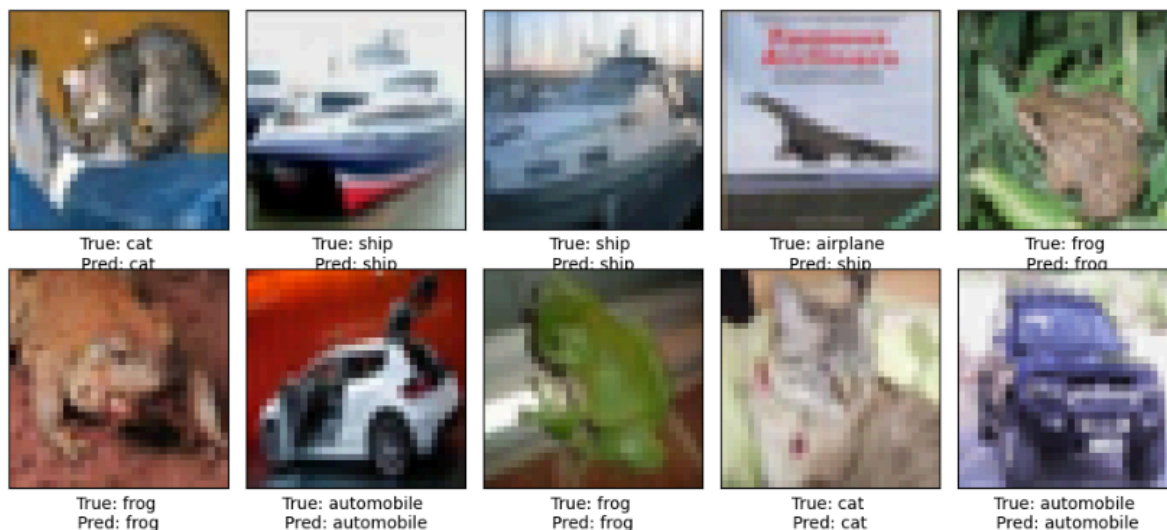
```
CustomCNN Test Accuracy: 0.4883
Showing predictions for CustomCNN:
/tmp/ipykernel_31/2183628123.py:37: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  plt.xlabel(f"True: {class_names[int(y_test[i])]} \nPred: {class_names[pred_label]}")
```

Predictions After Training - CustomCNN



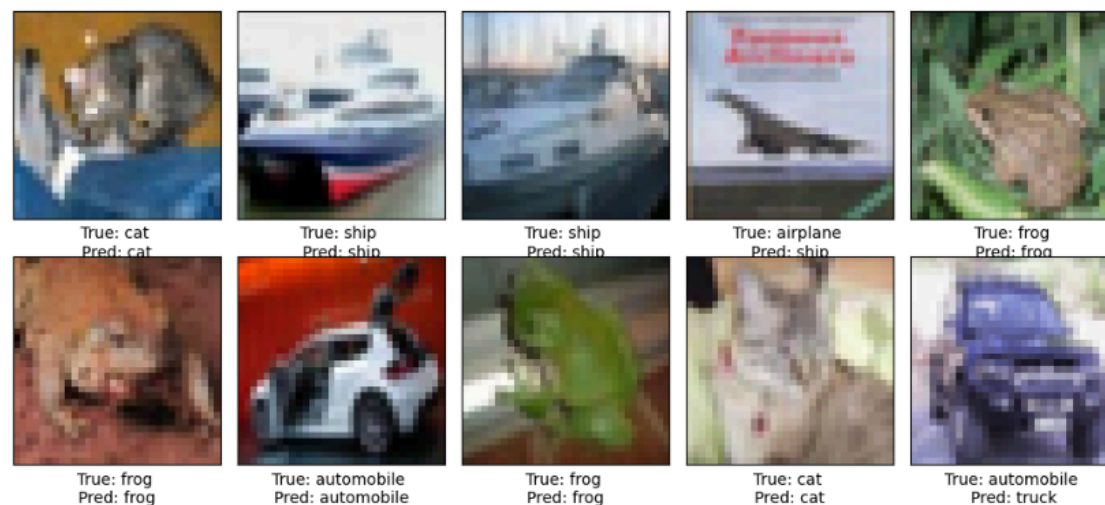
VGG16 Test Accuracy: 0.6392
Showing predictions for VGG16:

Predictions After Training - VGG16



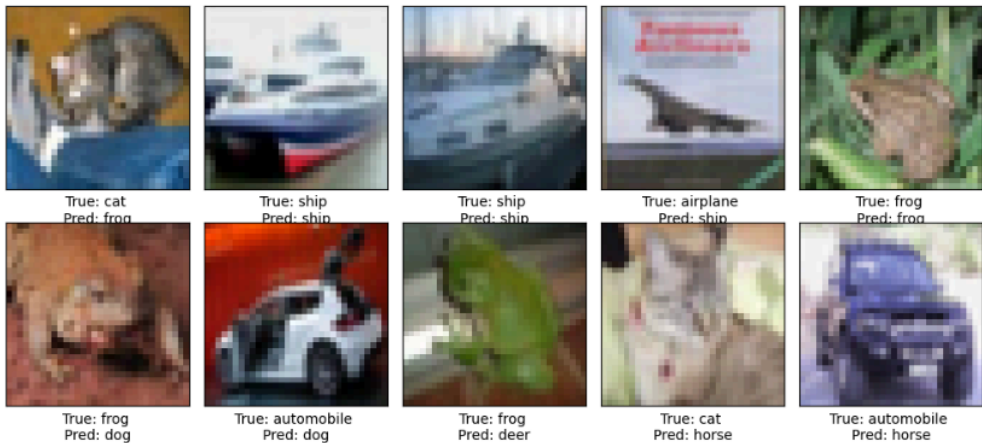
ResNet50 Test Accuracy: 0.6621
Showing predictions for ResNet50:

Predictions After Training - ResNet50



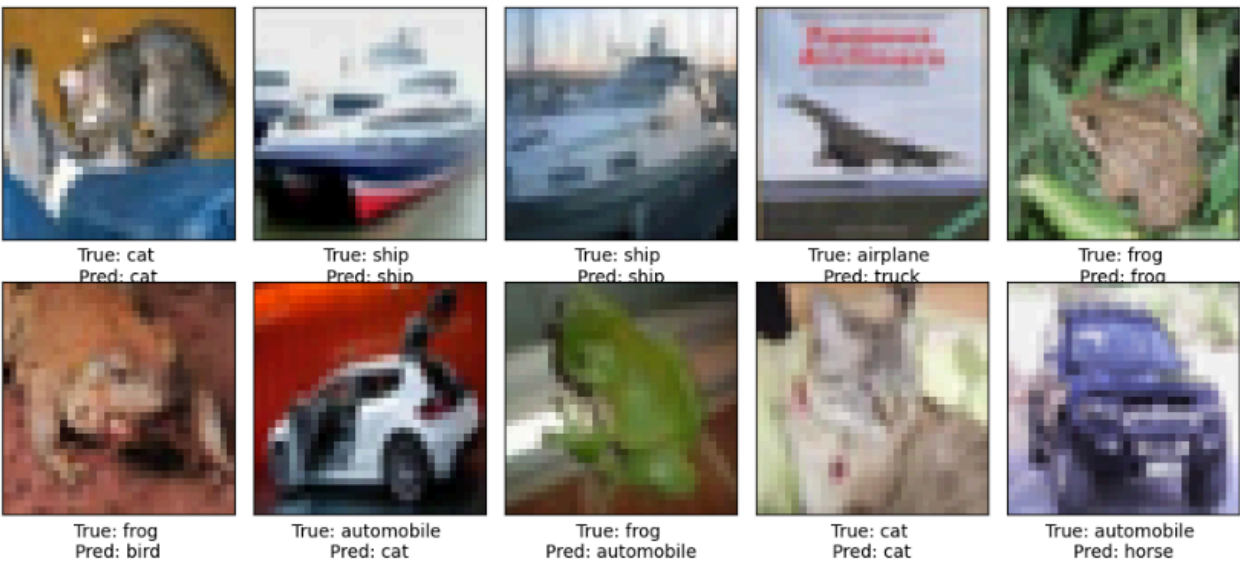
MobileNet Test Accuracy: 0.2280
Showing predictions for MobileNet:

Predictions After Training - MobileNet



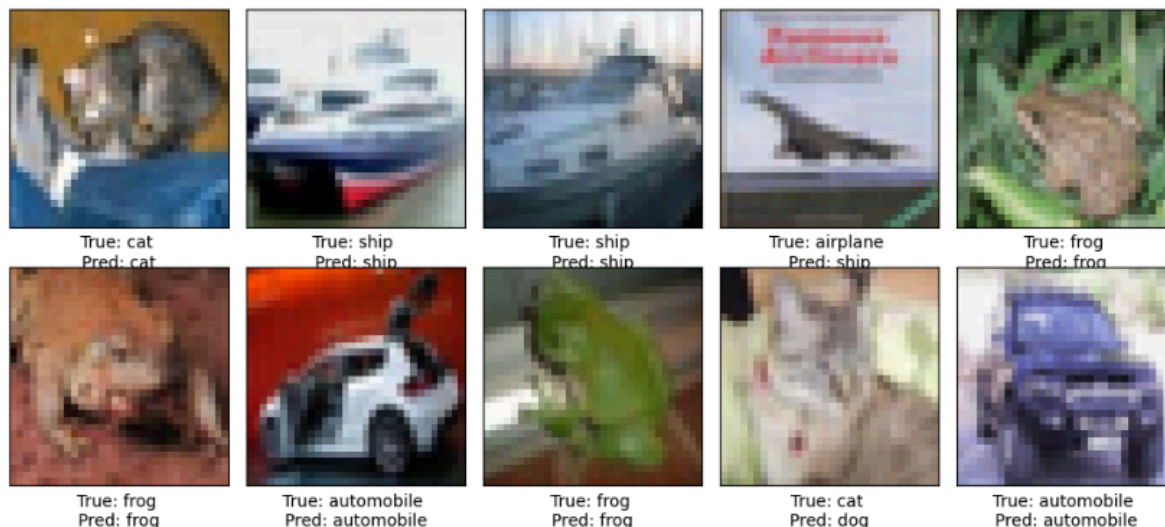
EfficientNetB1 Test Accuracy: 0.5427
Showing predictions for EfficientNetB1:

Predictions After Training - EfficientNetB1



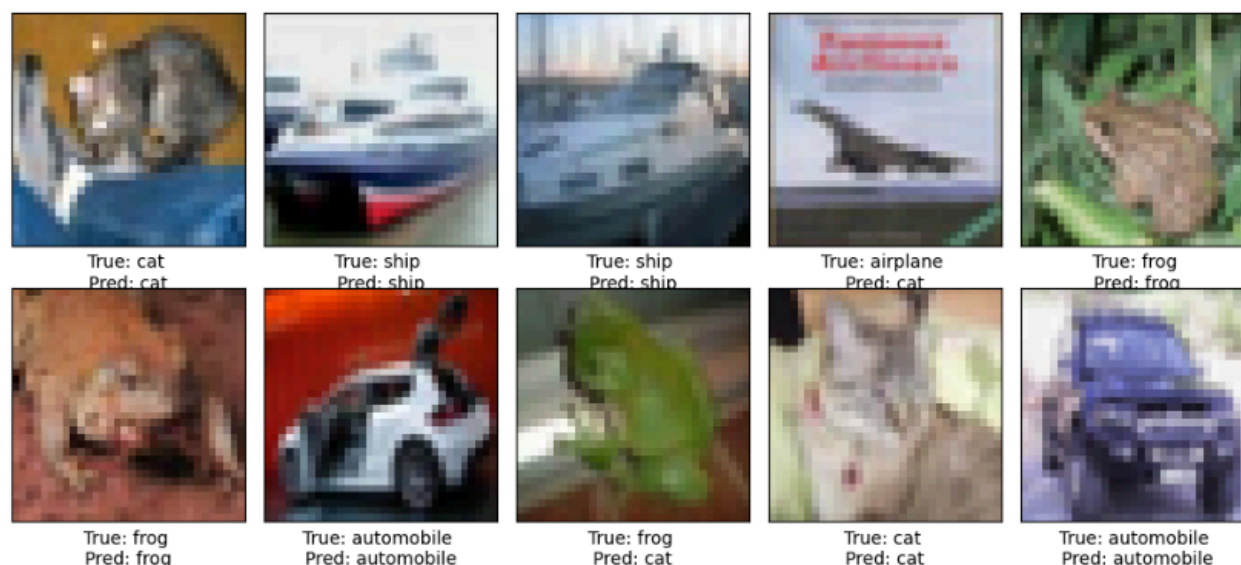
DenseNet121 Test Accuracy: 0.6790
 Showing predictions for DenseNet121:

Predictions After Training - DenseNet121



Xception Test Accuracy: 0.7724
 Showing predictions for Xception:

Predictions After Training - Xception



```
# 10. Final Results Summary
print("\n--- Final Test Accuracies ---")
for name, acc in results.items():
    print(f"{name}: {acc:.4f}")
```

```
--- Final Test Accuracies ---
CustomCNN: 0.4883
VGG16: 0.6392
ResNet50: 0.6621
MobileNet: 0.2280
EfficientNetB1: 0.5427
DenseNet121: 0.6790
Xception: 0.7724
```

Experiment-5

Aim : To implement Brain Tumor Segmentation using UNET Architecture.

Software Used : Jupyter Notebook

Dataset Used : Brain Tumor Segmentation

Theory :

Image segmentation involves classifying each pixel of an image into a category. In this experiment, we use the U-Net architecture, a popular deep learning model designed for biomedical image segmentation. U-Net has an encoder-decoder structure with skip connections that help preserve spatial information. It is particularly effective for tasks like brain tumor segmentation, where precise localization of the tumor region is critical. Model performance is typically evaluated using metrics like Dice coefficient, Intersection over Union (IoU), and pixel-wise accuracy.

Source Code :

```
import os
import zipfile
import numpy as np
import matplotlib.pyplot as plt
import cv2
import tensorflow as tf
from tensorflow.keras.layers import Conv2D, MaxPooling2D, UpSampling2D, Concatenate, Input
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split

# Define dataset path
zip_path = "../Deep learning labs/braintumordataset.zip"
extract_path = "/brain_tumor_segmentation"

# Extract dataset
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("✅ Dataset extracted successfully!")

# Define image and mask directories
image_dir = os.path.join(extract_path, "images")
mask_dir = os.path.join(extract_path, "masks")

# Check if folders exist
if not os.path.exists(image_dir) or not os.path.exists(mask_dir):
    raise Exception("❌ Image or Mask folder not found! Check extraction path.")
IMG_SIZE = 64 # Reduce image size for faster training
BATCH_SIZE = 2 # Reduce batch size to prevent crashes
```

```
# Function to Load & preprocess images
def load_data(image_dir, mask_dir, max_samples=300): # Reduce dataset size
    images, masks = [], []
    image_files = sorted(os.listdir(image_dir))[:max_samples] # Only Load 300 images
    mask_files = sorted(os.listdir(mask_dir))[:max_samples]

    for img_name, mask_name in zip(image_files, mask_files):
        img_path = os.path.join(image_dir, img_name)
        mask_path = os.path.join(mask_dir, mask_name)

        img = cv2.imread(img_path, cv2.IMREAD_COLOR)
        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE)) / 255.0 # Normalize
        images.append(img)

        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
        mask = cv2.resize(mask, (IMG_SIZE, IMG_SIZE)) / 255.0 # Normalize
        mask = np.expand_dims(mask, axis=-1) # Add channel dimension
        masks.append(mask)

    return np.array(images), np.array(masks)

# Load smaller dataset
X, Y = load_data(image_dir, mask_dir, max_samples=300)
print(f"✅ Dataset Loaded: Images: {X.shape}, Masks: {Y.shape}")

# Split dataset (reduce steps per epoch)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
print(f"📊 Training: {X_train.shape}, Testing: {X_test.shape}")
```

```

# Build a smaller UNet model
def build_unet(input_shape=(IMG_SIZE, IMG_SIZE, 3)):
    inputs = Input(input_shape)

    # Encoder (reduce filters)
    c1 = Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
    c1 = Conv2D(32, (3, 3), activation='relu', padding='same')(c1)
    p1 = MaxPooling2D((2, 2))(c1)

    c2 = Conv2D(64, (3, 3), activation='relu', padding='same')(p1)
    c2 = Conv2D(64, (3, 3), activation='relu', padding='same')(c2)
    p2 = MaxPooling2D((2, 2))(c2)

    c3 = Conv2D(128, (3, 3), activation='relu', padding='same')(p2)
    c3 = Conv2D(128, (3, 3), activation='relu', padding='same')(c3)
    p3 = MaxPooling2D((2, 2))(c3)

    # Bottleneck
    c4 = Conv2D(256, (3, 3), activation='relu', padding='same')(p3)
    c4 = Conv2D(256, (3, 3), activation='relu', padding='same')(c4)

    # Decoder
    u5 = UpSampling2D((2, 2))(c4)
    u5 = Concatenate()([u5, c3])
    c5 = Conv2D(128, (3, 3), activation='relu', padding='same')(u5)
    c5 = Conv2D(128, (3, 3), activation='relu', padding='same')(c5)

    u6 = UpSampling2D((2, 2))(c5)
    u6 = Concatenate()([u6, c2])
    c6 = Conv2D(64, (3, 3), activation='relu', padding='same')(u6)
    c6 = Conv2D(64, (3, 3), activation='relu', padding='same')(c6)

    u7 = UpSampling2D((2, 2))(c6)
    u7 = Concatenate()([u7, c1])
    c7 = Conv2D(32, (3, 3), activation='relu', padding='same')(u7)
    c7 = Conv2D(32, (3, 3), activation='relu', padding='same')(c7)

    outputs = Conv2D(1, (1, 1), activation='sigmoid')(c7)

    model = Model(inputs, outputs)
    return model

```

```

# Compile model
model = build_unet()
model.compile(optimizer=Adam(learning_rate=1e-4), loss="binary_crossentropy", metrics=["accuracy"])
model.summary()
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Reduce epochs & early stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2, min_lr=1e-6)

history = model.fit(
    X_train, Y_train,
    validation_data=(X_test, Y_test),
    epochs=10, # Reduced epochs for faster training
    batch_size=BATCH_SIZE,
    callbacks=[early_stopping, reduce_lr] # Apply early stopping & Learning rate reduction
)

def visualize_predictions(model, X_test, Y_test, num_samples=5):
    preds = model.predict(X_test)
    for i in range(num_samples):
        plt.figure(figsize=(12, 4))

        plt.subplot(1, 3, 1)
        plt.imshow(X_test[i])
        plt.title("Original Image")

        plt.subplot(1, 3, 2)
        plt.imshow(Y_test[i].squeeze(), cmap='gray')
        plt.title("Ground Truth Mask")

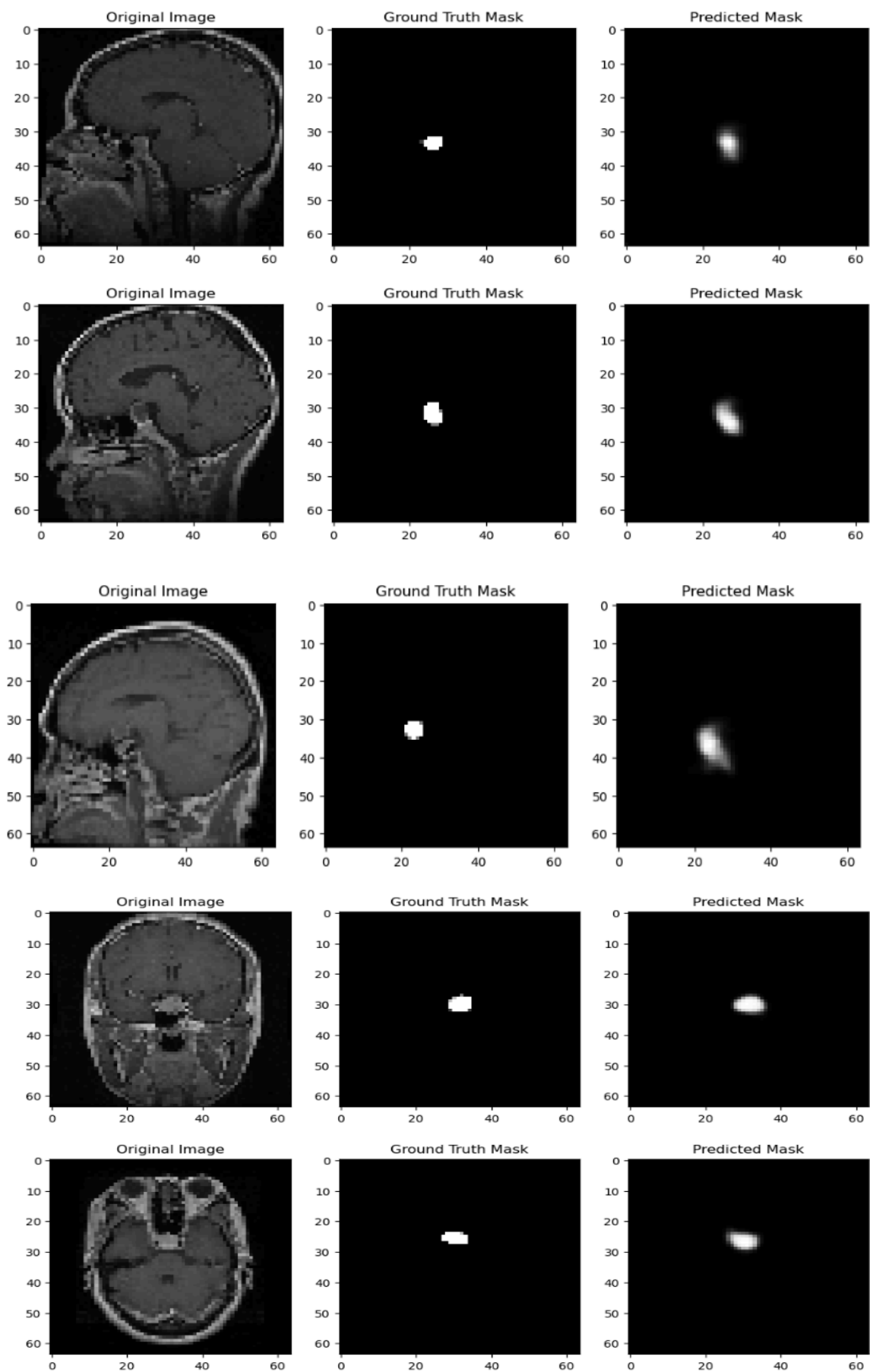
        plt.subplot(1, 3, 3)
        plt.imshow(preds[i].squeeze(), cmap='gray')
        plt.title("Predicted Mask")

    plt.show()

visualize_predictions(model, X_test, Y_test)

```

Output :

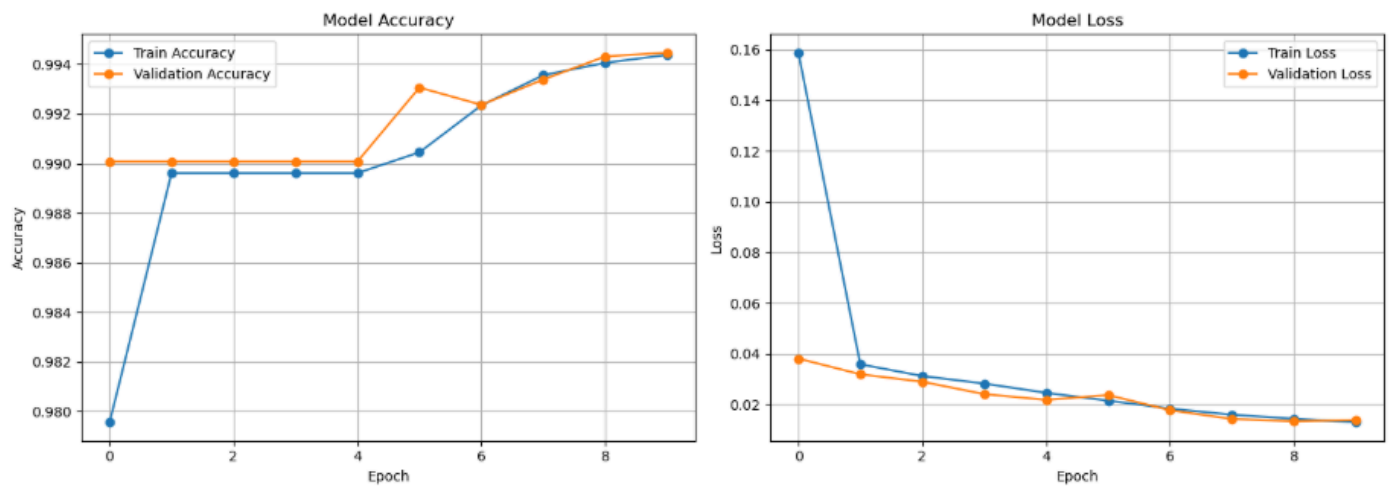


```
# Plot training & validation accuracy values
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', marker='o')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', marker='o')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', marker='o')
plt.plot(history.history['val_loss'], label='Validation Loss', marker='o')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Experiment-6

Aim : To implement Next word predictor using RNN, LSTM, BiLSTM, GRU, and Attention based (all variants).

Software Used : Google Colab

Theory :

Next word prediction is a sequence modeling task where the goal is to predict the most probable next word given a sequence of previous words. In this experiment, various deep learning models are used: RNN (Recurrent Neural Network) for basic sequence learning, LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) to handle long-term dependencies and mitigate vanishing gradient problems, BiLSTM (Bidirectional LSTM) to capture context from both past and future directions, and Attention-based models to focus on the most relevant parts of the input sequence. The models are trained on text datasets and evaluated using metrics like accuracy and perplexity.

Source Code :

```
# Imports
import numpy as np
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU, Bidirectional, Dense, Attention, Concatenate, Input
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.layers import GlobalAveragePooling1D

# Sample text data
texts = ["hello world", "machine learning is great", "next word prediction is fun"]

# Step 1: Preprocessing
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)
vocab_size = len(tokenizer.word_index) + 1

sequences = tokenizer.texts_to_sequences(texts)

X, y = [], []
for seq in sequences:
    for i in range(1, len(seq)):
        X.append(seq[:i])
        y.append(seq[i])

max_sequence_length = max([len(seq) for seq in X])
X = pad_sequences(X, maxlen=max_sequence_length, padding='pre')
y = to_categorical(y, num_classes=vocab_size)

print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")

# Parameters
embedding_dim = 50
units = 128
```

```

# Step 2: Build Models

### 1. Simple RNN Model
rnn_model = Sequential()
rnn_model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_sequence_length))
rnn_model.add(SimpleRNN(units))
rnn_model.add(Dense(vocab_size, activation='softmax'))
rnn_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
print("\nTraining Simple RNN Model...")
rnn_model.fit(X, y, epochs=10, batch_size=32)
print("\nSimple RNN Model Summary:")
rnn_model.summary()

### 2. LSTM Model
lstm_model = Sequential()
lstm_model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_sequence_length))
lstm_model.add(LSTM(units))
lstm_model.add(Dense(vocab_size, activation='softmax'))
lstm_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
print("\nTraining LSTM Model...")
lstm_model.fit(X, y, epochs=10, batch_size=32)
print("\nLSTM Model Summary:")
lstm_model.summary()

### 3. BiLSTM Model
bilstm_model = Sequential()
bilstm_model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_sequence_length))
bilstm_model.add(Bidirectional(LSTM(units)))
bilstm_model.add(Dense(vocab_size, activation='softmax'))
bilstm_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
print("\nTraining BiLSTM Model...")
bilstm_model.fit(X, y, epochs=10, batch_size=32)
print("\nBiLSTM Model Summary:")
bilstm_model.summary()

```



```

### 4. GRU Model
gru_model = Sequential()
gru_model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_sequence_length))
gru_model.add(GRU(units))
gru_model.add(Dense(vocab_size, activation='softmax'))
gru_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
print("\nTraining GRU Model...")
gru_model.fit(X, y, epochs=10, batch_size=32)
print("\nGRU Model Summary:")
gru_model.summary()

### 5. Attention Model
# Attention Model (Corrected)
inputs = Input(shape=(max_sequence_length,))
embedding = Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_sequence_length)(inputs)
lstm_out = LSTM(units, return_sequences=True)(embedding)
attention_out = Attention()([lstm_out, lstm_out])

# Add pooling layer to fix dimensions
pooled_out = GlobalAveragePooling1D()(attention_out)

dense_out = Dense(units, activation='relu')(pooled_out)
output = Dense(vocab_size, activation='softmax')(dense_out)

attention_model = Model(inputs, output)
attention_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
print("\nTraining Attention Model...")
attention_model.fit(X, y, epochs=10, batch_size=32)
print("\nAttention Model Summary:")
attention_model.summary()

# Prediction function
def predict_next_word(model, tokenizer, text, max_sequence_length):
    sequence = tokenizer.texts_to_sequences([text])
    sequence = pad_sequences(sequence, maxlen=max_sequence_length, padding='pre')
    prediction = model.predict(sequence, verbose=0)
    predicted_index = np.argmax(prediction, axis=-1)[0]
    return tokenizer.index_word.get(predicted_index, None)

# Input for prediction
input_text = "machine learning is"

print("\nPrediction Results:")
print(f"Simple RNN Prediction: '{input_text}' => '{predict_next_word(rnn_model, tokenizer, input_text, max_sequence_length)}'")
print(f"LSTM Prediction: '{input_text}' => '{predict_next_word(lstm_model, tokenizer, input_text, max_sequence_length)}'")
print(f"BiLSTM Prediction: '{input_text}' => '{predict_next_word(bilstm_model, tokenizer, input_text, max_sequence_length)}'")
print(f"GRU Prediction: '{input_text}' => '{predict_next_word(gru_model, tokenizer, input_text, max_sequence_length)}'")
print(f"Attention Model Prediction: '{input_text}' => '{predict_next_word(attention_model, tokenizer, input_text, max_sequence_length)}'")

```

Output:

Simple RNN Model Summary:

Model: "sequential_23"

Layer (type)	Output Shape	Param #
embedding_25 (Embedding)	(None, 4, 50)	550
simple_rnn_8 (SimpleRNN)	(None, 128)	22,912
dense_25 (Dense)	(None, 11)	1,419

Total params: 74,645 (291.59 KB)

Trainable params: 24,881 (97.19 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 49,764 (194.39 KB)

LSTM Model Summary:

Model: "sequential_24"

Layer (type)	Output Shape	Param #
embedding_26 (Embedding)	(None, 4, 50)	550
lstm_14 (LSTM)	(None, 128)	91,648
dense_26 (Dense)	(None, 11)	1,419

Total params: 280,853 (1.07 MB)

Trainable params: 93,617 (365.69 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 187,236 (731.39 KB)

BiLSTM Model Summary:

Model: "sequential_25"

Layer (type)	Output Shape	Param #
embedding_27 (Embedding)	(None, 4, 50)	550
bidirectional_3 (Bidirectional)	(None, 256)	183,296
dense_27 (Dense)	(None, 11)	2,827

Total params: 560,021 (2.14 MB)
Trainable params: 186,673 (729.19 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 373,348 (1.42 MB)

GRU Model Summary:

Model: "sequential_26"

Layer (type)	Output Shape	Param #
embedding_28 (Embedding)	(None, 4, 50)	550
gru_3 (GRU)	(None, 128)	69,120
dense_28 (Dense)	(None, 11)	1,419

Total params: 213,269 (833.09 KB)
Trainable params: 71,089 (277.69 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 142,180 (555.39 KB)

Attention Model Summary:

Model: "functional_19"

Layer (type)	Output Shape	Param #	Connected to
input_layer_23 (InputLayer)	(None, 4)	0	-
embedding_29 (Embedding)	(None, 4, 50)	550	input_layer_23[0][0]
lstm_16 (LSTM)	(None, 4, 128)	91,648	embedding_29[0][0]
attention_5 (Attention)	(None, 4, 128)	0	lstm_16[0][0], lstm_16[0][0]
global_average_pooling1d (GlobalAveragePooling1D)	(None, 128)	0	attention_5[0][0]
dense_29 (Dense)	(None, 128)	16,512	global_average_poolin...
dense_30 (Dense)	(None, 11)	1,419	dense_29[0][0]

Total params: 330,389 (1.26 MB)
Trainable params: 110,129 (430.19 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 220,260 (860.39 KB)

Prediction Results:

Simple RNN Prediction: 'machine learning is' => 'great'
LSTM Prediction: 'machine learning is' => 'is'
BiLSTM Prediction: 'machine learning is' => 'is'
GRU Prediction: 'machine learning is' => 'is'
Attention Model Prediction: 'machine learning is' => 'is'

Experiment-7

Aim : To implement Landslide detection using UNet (all variants).

Software Used : Google Colab

Dataset used : landslide4sense

Theory :

Landslide detection is a pixel-wise classification task that identifies regions affected by landslides in satellite or aerial images. In this experiment, we use U-Net and its variants (such as U-Net++, Attention U-Net, etc.), which are encoder-decoder architectures with skip connections to retain spatial information. These models are well-suited for segmentation tasks, especially when precise boundary detection is critical. The performance is evaluated using metrics like Dice coefficient, Intersection over Union (IoU), and pixel-wise accuracy.

SOURCE CODE AND OUTPUT:

```
import pandas as pd
import numpy as np
import h5py
import glob
import matplotlib.pyplot as plt
%matplotlib inline
import tensorflow as tf
```

```
from google.colab import drive
drive.mount("/content/gdrive")
```

Mounted at /content/gdrive

```
import os
os.chdir(r'/content/gdrive/MyDrive/Land Slide/dataset')
```

dataset preprocessing

```
# Testing the dataset
path_single = r"TrainData/img/image_2000.h5"
path_single_mask = r'TrainData/mask/mask_2000.h5'
```

```
TRAIN_PATH = r"TrainData/img/*.h5"
TRAIN_MASK = r'TrainData/mask/*.h5'

TRAIN_XX = np.zeros((3799, 128, 128, 6))
TRAIN_YY = np.zeros((3799, 128, 128, 1))
all_train = sorted(glob.glob(TRAIN_PATH))
all_mask = sorted(glob.glob(TRAIN_MASK))
```

```

for i, (img, mask) in enumerate(zip(all_train, all_mask)):
    print(i, img, mask)
    with h5py.File(img) as hdf:
        ls = list(hdf.keys())
        data = np.array(hdf.get('img'))

        # assign 0 for the nan value
        data[np.isnan(data)] = 0.000001

        # to normalize the data
        mid_rgb = data[:, :, 1:4].max() / 2.0
        mid_slope = data[:, :, 12].max() / 2.0
        mid_elevation = data[:, :, 13].max() / 2.0

        # ndvi calculation
        data_red = data[:, :, 3]
        data_nir = data[:, :, 7]
        data_ndvi = np.divide(data_nir - data_red, np.add(data_nir, data_red))

        # final array
        TRAIN_XX[i, :, :, 0] = 1 - data[:, :, 3] / mid_rgb #RED
        TRAIN_XX[i, :, :, 1] = 1 - data[:, :, 2] / mid_rgb #GREEN
        TRAIN_XX[i, :, :, 2] = 1 - data[:, :, 1] / mid_rgb #BLUE
        TRAIN_XX[i, :, :, 3] = data_ndvi #NDVI
        TRAIN_XX[i, :, :, 4] = 1 - data[:, :, 12] / mid_slope #SLOPE
        TRAIN_XX[i, :, :, 5] = 1 - data[:, :, 13] / mid_elevation #ELEVATION

    with h5py.File(mask) as hdf:
        ls = list(hdf.keys())
        data = np.array(hdf.get('mask'))
        TRAIN_YY[i, :, :, 0] = data

87 TrainData/img/image_1077.h5 TrainData/mask/mask_1077.h5
88 TrainData/img/image_1078.h5 TrainData/mask/mask_1078.h5
89 TrainData/img/image_1079.h5 TrainData/mask/mask_1079.h5
90 TrainData/img/image_108.h5 TrainData/mask/mask_108.h5
91 TrainData/img/image_1080.h5 TrainData/mask/mask_1080.h5
92 TrainData/img/image_1081.h5 TrainData/mask/mask_1081.h5
93 TrainData/img/image_1082.h5 TrainData/mask/mask_1082.h5
94 TrainData/img/image_1083.h5 TrainData/mask/mask_1083.h5
95 TrainData/img/image_1084.h5 TrainData/mask/mask_1084.h5
96 TrainData/img/image_1085.h5 TrainData/mask/mask_1085.h5

```

Testing min, max values in train data

```

# TRAIN_XX_n = TRAIN_XX / TRAIN_XX.max()
TRAIN_XX[np.isnan(TRAIN_XX)] = 0.000001
print(TRAIN_XX.min(), TRAIN_XX.max(), TRAIN_YY.min(), TRAIN_YY.max())

-1.0 1.0 0.0 1.0

```

Visualization of the training data

```

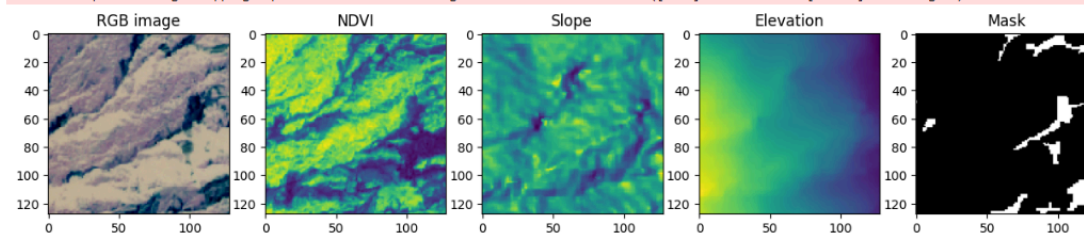
img = 234
fig, (ax1, ax2, ax3, ax4, ax5) = plt.subplots(1, 5, figsize=(15, 10))

ax1.set_title("RGB image")
ax2.set_title("NDVI")
ax3.set_title("Slope")
ax4.set_title("Elevation")
ax5.set_title("Mask")

ax1.imshow(TRAIN_XX[img, :, :, 0:3])
ax2.imshow(TRAIN_XX[img, :, :, 3])
ax3.imshow(TRAIN_XX[img, :, :, 4])
ax4.imshow(TRAIN_XX[img, :, :, 5])
ax5.imshow(TRAIN_YY[img, :, :, 0], cmap='gray')
plt.show()

```

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



train test split

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_temp, y_train, y_temp = train_test_split(TRAIN_XX, TRAIN_YY, test_size=0.3, shuffle=True)
x_validation, x_test, y_validation, y_test = train_test_split(x_temp, y_temp, test_size=0.5, shuffle=True)
```

```
x_train.shape, y_train.shape
```

```
((2659, 128, 128, 6), (2659, 128, 128, 1))
```

```
x_validation.shape, y_validation.shape
```

```
((570, 128, 128, 6), (570, 128, 128, 1))
```

```
x_test.shape, y_test.shape
```

```
((570, 128, 128, 6), (570, 128, 128, 1))
```

```
# to release some memory, delete the unnecessary variable
del TRAIN_XX
del TRAIN_YY
del all_train
del all_mask
```

```
from tensorflow.keras import backend as K
```

```
# recall
```

```
def recall_m(y_true, y_pred):
    true_positives = K.sum(K.round(K.clip(y_true * y_pred, 0, 1)))
    possible_positives = K.sum(K.round(K.clip(y_true, 0, 1)))
    recall = true_positives / (possible_positives + K.epsilon())
    return recall
```

```
# precision
```

```
def precision_m(y_true, y_pred):
    true_positives = K.sum(K.round(K.clip(y_true * y_pred, 0, 1)))
    predicted_positives = K.sum(K.round(K.clip(y_pred, 0, 1)))
    precision = true_positives / (predicted_positives + K.epsilon())
    return precision
```

```
#f1 score
```

```
def f1_m(y_true, y_pred):
    precision = precision_m(y_true, y_pred)
    recall = recall_m(y_true, y_pred)
    return 2*((precision*recall)/(precision+recall+K.epsilon()))
```

UNet

```
from tensorflow.keras.losses import binary_crossentropy

def unet_model(IMG_WIDTH, IMG_HEIGHT, IMG_CHANNELS):
    inputs = tf.keras.layers.Input((IMG_WIDTH, IMG_HEIGHT, IMG_CHANNELS))

    # Converted inputs to floating
    #s = tf.keras.layers.Lambda(lambda x: x / 255)(inputs)

    #Contraction path
    c1 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(inputs)
    c1 = tf.keras.layers.Dropout(0.1)(c1)
    c1 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c1)
    p1 = tf.keras.layers.MaxPooling2D((2, 2))(c1)

    c2 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p1)
    c2 = tf.keras.layers.Dropout(0.1)(c2)
    c2 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c2)
    p2 = tf.keras.layers.MaxPooling2D((2, 2))(c2)

    c3 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p2)
    c3 = tf.keras.layers.Dropout(0.2)(c3)
    c3 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c3)
    p3 = tf.keras.layers.MaxPooling2D((2, 2))(c3)

    c4 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p3)
    c4 = tf.keras.layers.Dropout(0.2)(c4)
    c4 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c4)
    p4 = tf.keras.layers.MaxPooling2D(pool_size=(2, 2))(c4)

    c5 = tf.keras.layers.Conv2D(256, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p4)
    c5 = tf.keras.layers.Dropout(0.3)(c5)
    c5 = tf.keras.layers.Conv2D(256, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c5)

    #Expansive path
    u6 = tf.keras.layers.Conv2DTranspose(128, (2, 2), strides=(2, 2), padding='same')(c5)
    u6 = tf.keras.layers.concatenate([u6, c4])
    c6 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u6)
    c6 = tf.keras.layers.Dropout(0.2)(c6)
    c6 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c6)

    u7 = tf.keras.layers.Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same')(c6)
    u7 = tf.keras.layers.concatenate([u7, c3])
    c7 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u7)
    c7 = tf.keras.layers.Dropout(0.2)(c7)
    c7 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c7)

    u8 = tf.keras.layers.Conv2DTranspose(32, (2, 2), strides=(2, 2), padding='same')(c7)
    u8 = tf.keras.layers.concatenate([u8, c2])
    c8 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u8)
    c8 = tf.keras.layers.Dropout(0.1)(c8)
    c8 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c8)

    u9 = tf.keras.layers.Conv2DTranspose(16, (2, 2), strides=(2, 2), padding='same')(c8)
    u9 = tf.keras.layers.concatenate([u9, c1], axis=3)
    c9 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u9)
    c9 = tf.keras.layers.Dropout(0.1)(c9)
    c9 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c9)

    outputs = tf.keras.layers.Conv2D(1, (1, 1), activation='sigmoid')(c9)

    model = tf.keras.Model(inputs=[inputs], outputs=[outputs])
    model.compile(optimizer='adam', loss=binary_crossentropy, metrics=['accuracy', 'f1_m', 'precision_m', 'recall_m'])

    return model

unet = unet_model(128, 128, 6)
# unet.summary()
checkpointer = tf.keras.callbacks.ModelCheckpoint("best_unet.h5", monitor="val_f1_m", verbose=1, save_best_only=True, mode="max")
# earlyStopping = tf.keras.callbacks.EarlyStopping(monitor='val_f1_m', patience=10, verbose=1, mode='max')

callbacks = [
    # earlyStopping,
    checkpointer
]
history = unet.fit(x_train, y_train, batch_size=16,
                  epochs=100,
                  verbose = 2,
                  validation_data=(x_validation, y_validation),
                  callbacks=callbacks)

unet.save("unet_save.h5")
```

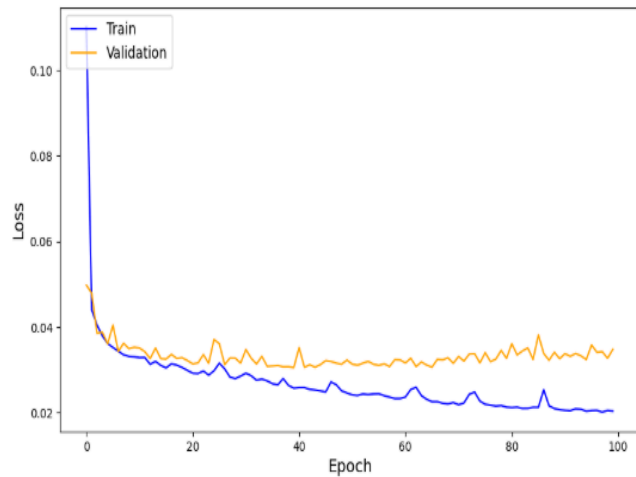
Prediction

```
# Create a figure and axis for the loss plot
plt.figure(figsize=(10, 6)) # Increased the figure size

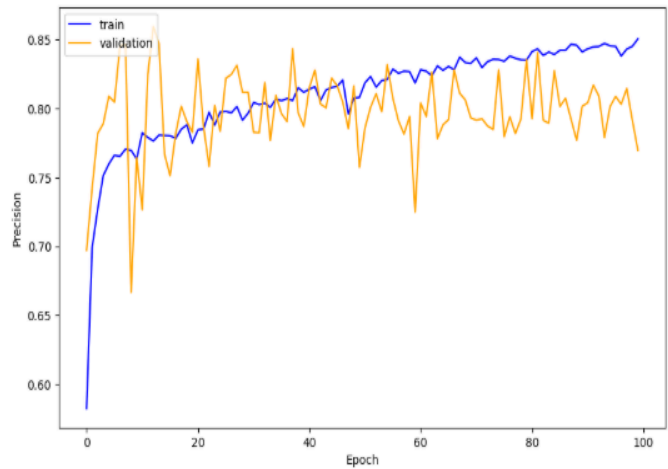
# Plot training and validation loss
plt.plot(history.history['loss'], label='Train', color='blue')
plt.plot(history.history['val_loss'], label='Validation', color='orange')

# Set Labels and Legend
plt.xlabel('Epoch', fontsize=14) # Increased the font size
plt.ylabel('Loss', fontsize=14) # Increased the font size
plt.title('Training and Validation Loss Over Epochs', fontsize=16) # Increased the font size
plt.legend(loc='upper left', fontsize=12) # Increased the font size

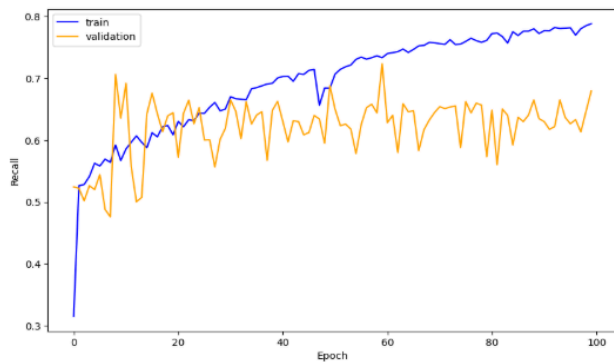
# Display the plot
plt.show()
```



```
# Create a figure and axis for the precision plot
plt.figure(figsize=(10, 6))
plt.plot(history.history['precision_m'], label='train', color='blue')
plt.plot(history.history['val_precision_m'], label='validation', color='orange')
plt.xlabel('Epoch')
plt.ylabel('Precision')
plt.legend(loc='upper left')
plt.show()
```



```
# Create a figure and axis for the recall plot
plt.figure(figsize=(10, 6))
plt.plot(history.history['recall_m'], label='train', color='blue')
plt.plot(history.history['val_recall_m'], label='validation', color='orange')
plt.xlabel('Epoch')
plt.ylabel('Recall')
plt.legend(loc='upper left')
plt.show()
```

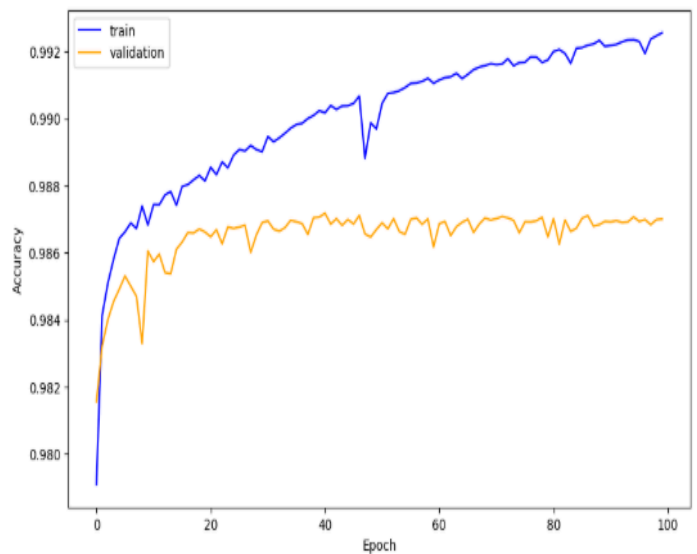


```
import matplotlib.pyplot as plt

# Create a figure and axis for the accuracy plot
plt.figure(figsize=(10, 6))
plt.plot(history.history['accuracy'], label='train', color='blue')
plt.plot(history.history['val_accuracy'], label='validation', color='orange')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(loc='upper left')

# Format the y-axis to show up to two decimal places
plt.gca().yaxis.set_major_formatter('{:.3f}'.format)

plt.show()
```

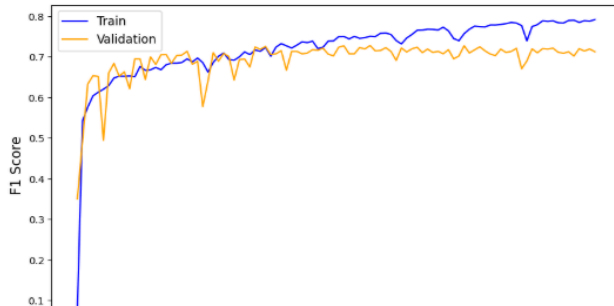


```
# Create a figure and axis for the F1 score plot
plt.figure(figsize=(10, 6)) # Increased the figure size

# Plot training and validation F1 score
plt.plot(history.history['f1_m'], label='Train', color='blue')
plt.plot(history.history['val_f1_m'], label='Validation', color='orange')

# Set Labels and Legend
plt.xlabel('Epoch', fontsize=14) # Increased the font size
plt.ylabel('F1 Score', fontsize=14) # Increased the font size
plt.title('Training and Validation F1 Score Over Epochs', fontsize=16) # Increased the font size
plt.legend(loc='upper left', fontsize=12) # Increased the font size

# Display the plot
plt.show()
```




```
import matplotlib.pyplot as plt

# Determine the number of test images
num_test_images = x_test.shape[0]

for i in range(num_test_images):
    fig, (ax1, ax2, ax3, ax4, ax5, ax6) = plt.subplots(1, 6, figsize=(20, 10))

    # Display RGB image
    ax1.imshow(x_test[i, :, :, 0:3])
    ax1.axis('off')

    # Display NDVI
    ax2.imshow(x_test[i, :, :, 3])
    ax2.axis('off')

    # Display Slope
    ax3.imshow(x_test[i, :, :, 4])
    ax3.axis('off')

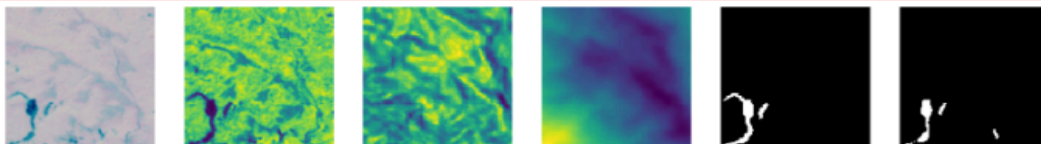
    # Display Elevation
    ax4.imshow(x_test[i, :, :, 5])
    ax4.axis('off')

    # Display Mask in black and white
    ax5.imshow(y_test[i, :, :, 0], cmap='gray')
    ax5.axis('off')

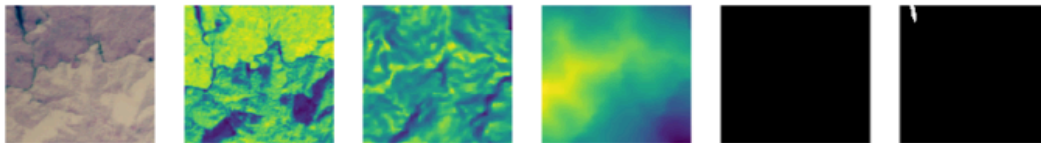
    # Display Prediction in black and white
    ax6.imshow(pred_img[i, :, :, 0], cmap='gray')
    ax6.axis('off')

plt.show()
```

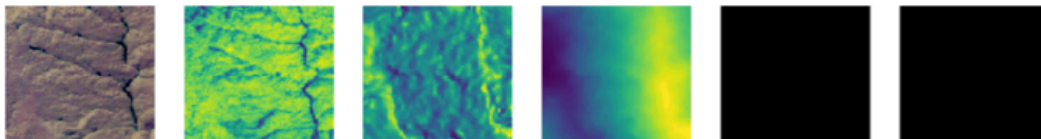
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



```
import matplotlib.pyplot as plt

img = 255
fig, (ax1, ax2, ax3, ax4, ax5, ax6) = plt.subplots(1, 6, figsize=(20, 10))

# Display RGB image
ax1.imshow(x_test[img, :, :, 0:3])
ax1.set_title('RGB Image')

# Display NDVI
ax2.imshow(x_test[img, :, :, 3])
ax2.set_title('NDVI')

# Display Slope
ax3.imshow(x_test[img, :, :, 4])
ax3.set_title('Slope')

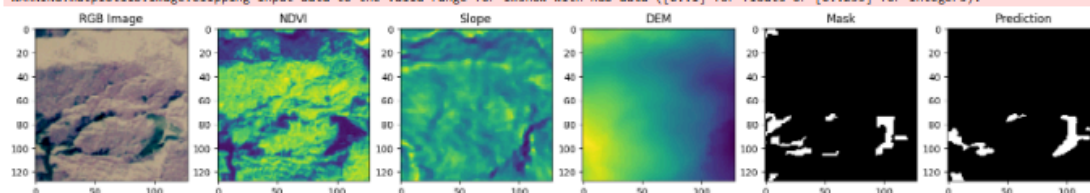
# Display Elevation
ax4.imshow(x_test[img, :, :, 5])
ax4.set_title('DEM')

# Display Mask in black and white
ax5.imshow(y_test[img, :, :, 0], cmap='gray') # Using a grayscale colormap for Mask
ax5.set_title('Mask')

# Display Prediction in black and white
ax6.imshow(pred_img[img, :, :, 0], cmap='gray') # Using a grayscale colormap for Prediction
ax6.set_title('Prediction')

plt.show()
```

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



Experiment-8

Aim : To implement Text summarization using transformer.

Software used : Google Colab

Theory :

Text summarization aims to generate a concise and coherent version of a longer document while preserving its key information. In this experiment, we use the Transformer architecture, which relies on self-attention mechanisms to model relationships between words in a sequence without recurrent layers. Transformers enable efficient handling of long texts and are the foundation for powerful models like BERT and T5. The model is trained or fine-tuned to perform abstractive summarization, generating human-like summaries, and evaluated using metrics like ROUGE and BLEU scores.

Source code and output :

```
import torch
from transformers import pipeline

# Initialize the summarization pipeline
summarizer = pipeline("summarization", model="facebook/bart-large-cnn", device=0 if torch.cuda.is_available() else -1)
input_text = """
The rapid advancement of artificial intelligence (AI) has transformed various industries, including healthcare, finance, and education. AI systems can now diagnose diseases with accuracy comparable to human doctors, automate financial trading, and personalize learning. However, these advancements come with challenges. Ethical concerns, such as bias in AI algorithms and job displacement due to automation, are being addressed. Additionally, the energy consumption of large AI models raises environmental concerns. Governments and organizations are working to ensure AI is developed and used responsibly.
"""

def summarize_text(text, max_length=130, min_length=30, do_sample=False):
    # Summarizes the input text using a Transformer-based model.
    summary = summarizer(
        text,
        max_length=max_length,
        min_length=min_length,
        do_sample=do_sample,
        truncation=True
    )[0]['summary_text']
    return summary

# Save summary to a file
def save_summary(summary, output_file="summary.txt"):
    with open(output_file, 'w', encoding='utf-8') as f:
        f.write(summary)
    print(f"Summary saved to {output_file}")

# Main execution
if __name__ == "__main__":
    print("Generating summary...")
    summary = summarize_text(input_text)
    print("Summary:")
    print(summary)
    save_summary(summary)
    print("Summarization completed.")
```

Output :

```
summary.txt X ...
1 The rapid advancement of artificial intelligence has transformed various
2 industries, including healthcare, finance, and education. Ethical concerns,
3 such as bias in AI algorithms and job displacement due to automation,
4 are significant hurdles. Governments and organizations are working to
5 establish regulations to ensure AI is used responsibly while fostering
6 innovation.
```

Experiment-9

Aim : To implement Plant disease prediction using swin and vision transformer.

Software Used : Jupyter Notebook

Dataset Used : Sugarcane Leaf Disease Dataset

Theory :

Plant disease prediction involves classifying images of plant leaves to detect and identify diseases. In this experiment, Swin Transformer and Vision Transformer (ViT) models are used. These are transformer-based architectures that process images by treating them as sequences of patches rather than pixels. Vision Transformer captures global relationships between patches, while Swin Transformer introduces hierarchical feature learning with shifted windows, making it efficient for dense prediction tasks. Model performance is evaluated using accuracy, precision, recall, and F1 score.

Source Code and Output :

```
import os
import torch
import torch.nn as nn
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader, random_split
import timm
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
import numpy as np
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: cuda

```
DATA_DIR = "/kaggle/input/sugarcane-leaf-disease-dataset/Sugarcane Leaf Disease Dataset" # Path to your dataset folder
BATCH_SIZE = 16
IMAGE_SIZE = 224
NUM_EPOCHS = 10
NUM_CLASSES = 5
LEARNING_RATE = 1e-4
USE_VIT = False # Set to True to use Vision Transformer, False for Swin
```

```
transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3)
])
```

```
dataset = datasets.ImageFolder(root=DATA_DIR, transform=transform)
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(dataset, [train_size, val_size])

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE)
```

```
model_name = 'vit_base_patch16_224' if USE_VIT else 'swin_tiny_patch4_window7_224'
model = timm.create_model(model_name, pretrained=True, num_classes=NUM_CLASSES)
model.to(device)
print(f"Using model: {model_name}")

model.safetensors: 0%|          | 0.00/346M [00:00<?, ?B/s]
Using model: vit_base_patch16_224
```

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

```
def train_model():
    for epoch in range(NUM_EPOCHS):
        model.train()
        running_loss = 0.0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

        avg_loss = running_loss / len(train_loader)
        print(f"Epoch [{epoch+1}/{NUM_EPOCHS}], Loss: {avg_loss:.4f}")

        evaluate_model()
```

```
def evaluate_model():
    model.eval()
    correct = 0
    total = 0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    acc = 100 * correct / total
    print(f"Validation Accuracy: {acc:.2f}%")
    print(classification_report(all_labels, all_preds, target_names=dataset.classes))
```

```
def visualize_predictions():
    model.eval()
    class_names = dataset.classes
    images, labels = next(iter(val_loader))
    images = images.to(device)
    outputs = model(images)
    _, preds = torch.max(outputs, 1)

    plt.figure(figsize=(12, 8))
    for i in range(6):
        ax = plt.subplot(2, 3, i+1)
        img = images[i].cpu().permute(1, 2, 0).numpy()
        img = (img * 0.5) + 0.5 # unnormalize
        plt.imshow(img)
        true_label = class_names[labels[i]]
        predicted_label = class_names[preds[i]]
        plt.title(f"True: {true_label}\nPred: {predicted_label}")
        plt.axis("off")
    plt.tight_layout()
    plt.show()
```

```
train_model()
visualize_predictions()
```

```
DATA_DIR = "/kaggle/input/sugarcane-leaf-disease-dataset/Sugarcane Leaf Disease Dataset" # Path to your dataset folder
BATCH_SIZE = 16
IMAGE_SIZE = 224
NUM_EPOCHS = 10
NUM_CLASSES = 5
LEARNING_RATE = 1e-4
USE_VIT = True # Set to True to use Vision Transformer, False for Swin
```

```
train_model()
visualize_predictions()
```



```
Epoch [1/10], Loss: 0.6742
Validation Accuracy: 72.67%
      precision    recall  f1-score   support

Healthy      0.98      0.55      0.71      118
Mosaic       0.81      0.56      0.66       90
RedRot       0.92      0.70      0.79       93
Rust         1.00      0.85      0.92      110
Yellow       0.44      1.00      0.61       94

accuracy          0.73      505
macro avg         0.83      0.73      0.74      505
weighted avg      0.84      0.73      0.74      505
```

```
Epoch [2/10], Loss: 0.2359
Validation Accuracy: 88.12%
      precision    recall  f1-score   support

Healthy      0.90      0.87      0.89      118
Mosaic       0.91      0.70      0.79       90
RedRot       0.84      0.98      0.91       93
Rust         0.98      0.85      0.91      110
Yellow       0.80      1.00      0.89       94

accuracy          0.88      505
macro avg         0.89      0.88      0.88      505
weighted avg      0.89      0.88      0.88      505
```

```
Epoch [3/10], Loss: 0.1703
Validation Accuracy: 95.05%
      precision    recall  f1-score   support

Healthy      0.99      0.87      0.93      118
Mosaic       0.86      0.94      0.90       90
RedRot       0.93      1.00      0.96       93
Rust         0.99      0.95      0.97      110
Yellow       0.98      1.00      0.99       94

accuracy          0.95      505
macro avg         0.95      0.95      0.95      505
weighted avg      0.95      0.95      0.95      505
```

True: Rust
Pred: Rust



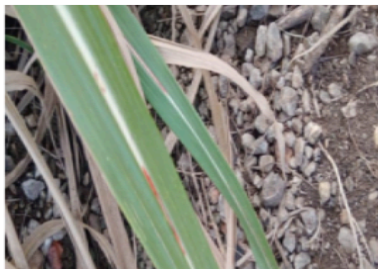
True: Healthy
Pred: Healthy



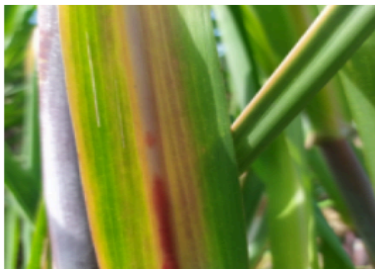
True: Mosaic
Pred: Healthy



True: RedRot
Pred: RedRot



True: Yellow
Pred: Yellow



True: Yellow
Pred: Yellow



Experiment-10

Aim : To Generate fake images using GAN.

Software Used : Google Colab

Dataset used : CelebA

Theory :

Generative Adversarial Networks (GANs) are a class of deep learning models used to generate realistic fake data, such as images. A GAN consists of two neural networks: a Generator, which creates fake images, and a Discriminator, which tries to distinguish between real and fake images. Both networks are trained adversarially, improving each other over time. In this experiment, GANs are used to generate fake images that resemble real ones, and model performance is evaluated qualitatively (by visual inspection) and quantitatively using metrics like Inception Score (IS) and Frechet Inception Distance (FID).

Source Code and Output :

```
# imports
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision.utils import make_grid
from torchvision.datasets import ImageFolder
import torchvision.transforms as T
import matplotlib.pyplot as plt
%matplotlib inline

import os

DATA_DIR = '../input/celebrities-100k/100k/'
print(len(os.listdir(DATA_DIR+'100k'))))

100000

print(os.listdir(DATA_DIR+'100k')[:10])

['168202.jpg', '059339.jpg', '039337.jpg', '102631.jpg', '042195.jpg', '157069.jpg', '011035.jpg', '163198.jpg', '104821.jpg', '099103.jpg']

image_size = 64
batch_size = 128
stats = (0.5, 0.5, 0.5), (0.5, 0.5, 0.5) # mean, std for normalize images

train_ds = ImageFolder(root=DATA_DIR,
                        transform=T.Compose([T.Resize(image_size),
                                             T.CenterCrop(image_size), # pick central square crop of it
                                             T.ToTensor(),
                                             T.Normalize(*stats)      # normalize => -1 to 1
                                             ]))

train_dl = DataLoader(train_ds, batch_size, shuffle=True, num_workers=3, pin_memory=True) # use multiple cores

def denorm(img_tensors):
    "Denormalize image tensor with specified mean and std"
    return img_tensors * stats[1][0] + stats[0][0]

def show_images(images, nmax=64):
    fig, ax = plt.subplots(figsize=(8,8))
    ax.set_xticks([]); ax.set_yticks([])
    ax.imshow(make_grid(denorm(images.detach()[:nmax]), nrow=8).permute(1, 2, 0))

def show_batch(dl, nmax=64):
    for images, _ in dl:
        show_images(images, nmax)
        break
```

```
def get_default_device():
    """Pick GPU if available, else CPU"""
    """ 3 things:
    1. Connected to Nvidia GPU
    2. Cuda drivers
    3. Pytorch suitable to GPU version
    then torch.cuda.is_available is True
    """
    if torch.cuda.is_available():
        return torch.device('cuda')
    else:
        return torch.device('cpu')

def to_device(data, device):
    """Move tensor(s) to chosen device"""
    if isinstance(data, (list,tuple)):
        return [to_device(x, device) for x in data]
    return data.to(device, non_blocking=True)

class DeviceDataLoader():
    """Wrap a dataloader to move data to a device"""
    def __init__(self, dl, device):
        self.dl = dl
        self.device = device

    def __iter__(self):
        """Yield a batch of data after moving it to device"""
        for b in self.dl:
            yield to_device(b, self.device)

    def __len__(self):
        """Number of batches"""
        return len(self.dl)
```

```
device = get_default_device()
```

```
train_dl = DeviceDataLoader(train_dl, device)
```

```
discriminator = nn.Sequential(
    # in: 3x 64 x 64
    nn.Conv2d(3, 64, kernel_size=4, stride=2, padding=1, bias=False),
    nn.BatchNorm2d(64),
    nn.LeakyReLU(0.2, inplace=True),
    # out: 64 x 32 x 32

    nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1, bias=False),
    nn.BatchNorm2d(128),
    nn.LeakyReLU(0.2, inplace=True),
    # out: 128 x 16 x 16

    nn.Conv2d(128, 256, kernel_size=4, stride=2, padding=1, bias=False),
    nn.BatchNorm2d(256),
    nn.LeakyReLU(0.2, inplace=True),
    # out: 256 x 8 x 8

    nn.Conv2d(256, 512, kernel_size=4, stride=2, padding=1, bias=False),
    nn.BatchNorm2d(512),
    nn.LeakyReLU(0.2, inplace=True),
    # out: 512 x 4 x 4

    nn.Conv2d(512, 1, kernel_size=4, stride=1, padding=0, bias=False),
    # out: 1 x 1 x 1

    nn.Flatten(),
    nn.Sigmoid()
)
```

```
discriminator = to_device(discriminator, device)
```

```
# create a tensor Batch_Size,C,H,W
x = torch.rand(size=(1, 3, 64, 64), dtype=torch.float32, device=device)
for layer in discriminator:
    x = layer(x)
    print(layer.__class__.__name__, 'output shape: \t',x.shape)
```

```
Conv2d output shape:      torch.Size([1, 64, 32, 32])
BatchNorm2d output shape: torch.Size([1, 64, 32, 32])
LeakyReLU output shape:  torch.Size([1, 64, 32, 32])
Conv2d output shape:      torch.Size([1, 128, 16, 16])
BatchNorm2d output shape: torch.Size([1, 128, 16, 16])
LeakyReLU output shape:  torch.Size([1, 128, 16, 16])
Conv2d output shape:      torch.Size([1, 256, 8, 8])
BatchNorm2d output shape: torch.Size([1, 256, 8, 8])
LeakyReLU output shape:  torch.Size([1, 256, 8, 8])
Conv2d output shape:      torch.Size([1, 512, 4, 4])
BatchNorm2d output shape: torch.Size([1, 512, 4, 4])
LeakyReLU output shape:  torch.Size([1, 512, 4, 4])
Conv2d output shape:      torch.Size([1, 1, 1, 1])
Flatten output shape:     torch.Size([1, 1])
Sigmoid output shape:     torch.Size([1, 1])
```

```
latent_size = 128
```

```
generator = nn.Sequential(
    # in: latent_size x 1 x 1

    nn.ConvTranspose2d(latent_size, 512, kernel_size=4, stride=1, padding=0, bias=False),
    nn.BatchNorm2d(512),
    nn.ReLU(True),
    # out: 512 x 4 x 4

    nn.ConvTranspose2d(512, 256, kernel_size=4, stride=2, padding=1, bias=False),
    nn.BatchNorm2d(256),
    nn.ReLU(True),
    # out: 256 x 8 x 8

    nn.ConvTranspose2d(256, 128, kernel_size=4, stride=2, padding=1, bias=False),
    nn.BatchNorm2d(128),
    nn.ReLU(True),
    # out: 128 x 16 x 16

    nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1, bias=False),
    nn.BatchNorm2d(64),
    nn.ReLU(True),
    # out: 64 x 32 x 32

    nn.ConvTranspose2d(64, 3, kernel_size=4, stride=2, padding=1, bias=False),
    nn.Tanh() # output is between -1 to 1
    # out: 3 x 64 x 64
)
```

```
x = torch.randn(size=(1, 128, 1, 1))
for layer in generator:
    x = layer(x)
    print(layer.__class__.__name__, 'output shape: \t',x.shape)
```

```
ConvTranspose2d output shape: torch.Size([1, 512, 4, 4])
BatchNorm2d output shape:   torch.Size([1, 512, 4, 4])
ReLU output shape:          torch.Size([1, 512, 4, 4])
ConvTranspose2d output shape: torch.Size([1, 256, 8, 8])
BatchNorm2d output shape:   torch.Size([1, 256, 8, 8])
ReLU output shape:          torch.Size([1, 256, 8, 8])
ConvTranspose2d output shape: torch.Size([1, 128, 16, 16])
BatchNorm2d output shape:   torch.Size([1, 128, 16, 16])
ReLU output shape:          torch.Size([1, 128, 16, 16])
ConvTranspose2d output shape: torch.Size([1, 64, 32, 32])
BatchNorm2d output shape:   torch.Size([1, 64, 32, 32])
ReLU output shape:          torch.Size([1, 64, 32, 32])
ConvTranspose2d output shape: torch.Size([1, 3, 64, 64])
Tanh output shape:          torch.Size([1, 3, 64, 64])
```



```
xb = torch.randn(batch_size, latent_size, 1, 1) # random Latent tensors
fake_images = generator(xb)
print(fake_images.shape)
show_images(fake_images)

torch.Size([128, 3, 64, 64])
```

```
def train_discriminator(real_images, opt_d):
    # Clear discriminator gradients
    opt_d.zero_grad()

    # Pass real images through discriminator
    real_preds = discriminator(real_images)
    real_targets = torch.ones(real_images.size(0), 1, device=device)
    real_loss = F.binary_cross_entropy(real_preds, real_targets)
    real_score = torch.mean(real_preds).item()

    # Generate fake images
    latent = torch.randn(batch_size, latent_size, 1, 1, device=device)
    fake_images = generator(latent)

    # Pass Fake images through discriminator
    fake_targets = torch.zeros(fake_images.size(0), 1, device=device)
    fake_preds = discriminator(fake_images)
    fake_loss = F.binary_cross_entropy(fake_preds, fake_targets)
    fake_score = torch.mean(fake_preds).item()

    # Update discriminator weights
    loss = real_loss + fake_loss
    loss.backward()
    opt_d.step()
    return loss.item(), real_score, fake_score
```

```
def train_generator(opt_g):
    # Clear generator gradients
    opt_g.zero_grad()

    # Generate fake images
    latent = torch.randn(batch_size, latent_size, 1, 1, device=device)
    fake_images = generator(latent)

    # Try to fool the discriminator
    preds = discriminator(fake_images)
    targets = torch.ones(batch_size, 1, device=device)
    loss = F.binary_cross_entropy(preds, targets)

    # Update generator
    loss.backward()
    opt_g.step()

    return loss.item()
```

```
from torchvision.utils import save_image
```

```
sample_dir = 'generated'
os.makedirs(sample_dir, exist_ok=True)
```

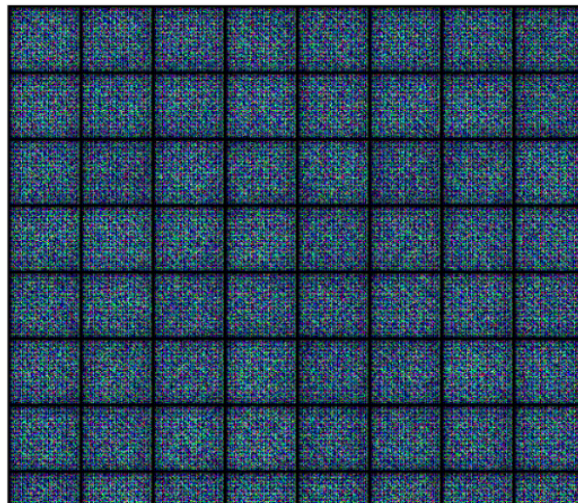
```
def save_samples(index, latent_tensors, show=True):
    fake_images = generator(latent_tensors)
    fake_fname = 'generated-images-{:0:4d}.png'.format(index)
    save_image(denorm(fake_images), os.path.join(sample_dir, fake_fname), nrow=8)
    print("Saving", fake_fname)

    if show:
        fig, ax = plt.subplots(figsize=(8,8))
        ax.set_xticks([]); ax.set_yticks([])
        ax.imshow(make_grid(fake_images.cpu().detach(), nrow=8).permute(1, 2, 0))
```

```
fixed_latent = torch.randn(64, latent_size, 1, 1, device=device)
```

```
save_samples(0, fixed_latent)
```

Saving generated-images-0000.png



```
from tqdm.notebook import tqdm
import torch.nn.functional as F
```

```
def fit(epochs, lr, start_idx = 1):
    torch.cuda.empty_cache()

    # Losses & scores
    losses_g = []
    losses_d = []
    real_scores = []
    fake_scores = []

    # Create optimizers
    opt_d = torch.optim.Adam(discriminator.parameters(), lr=lr, betas=(0.5, 0.999))
    opt_g = torch.optim.Adam(generator.parameters(), lr=lr, betas=(0.5, 0.999))

    for epoch in range(epochs):
        for real_images, _ in tqdm(train_dl):
            # Train discriminator
            loss_d, real_score, fake_score = train_discriminator(real_images, opt_d)
            # Train generator
            loss_g = train_generator(opt_g)

            # Record Losses & scores
            losses_g.append(loss_g)
            losses_d.append(loss_d)
            real_scores.append(real_score)
            fake_scores.append(fake_score)

            # Log Losses & scores (Last batch)
            print("epoch [{}]/[{}], loss_g: {:.4f}, loss_d: {:.4f}, real_score: {:.4f}, fake_score: {:.4f}".format(epoch+1, epochs, loss_g, loss_d, real_score, fake_score))
            # Save generated images
            save_samples(epoch+start_idx, fixed_latent, show=False)

    return losses_g, losses_d, real_scores, fake_scores
```

```
##Hyperparameters
lr = 0.00025
epochs = 50
```

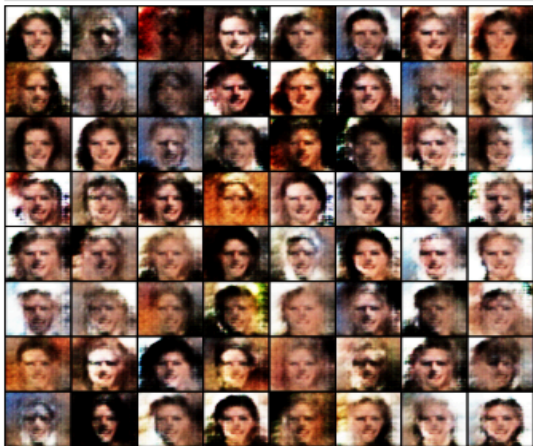
```
history = fit(epochs, lr)
```

```
# Save the model checkpoints
torch.save(generator.state_dict(), 'G.pth')
torch.save(discriminator.state_dict(), 'D.pth')
```

```
losses_g, losses_d, real_scores, fake_scores = history
```

```
from IPython.display import Image
```

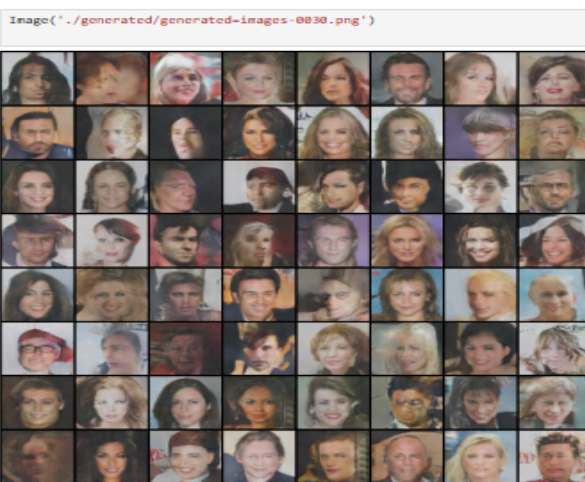
```
Image('./generated/generated-images-0001.png')
```



```
Image('./generated/generated-images-0005.png')
```



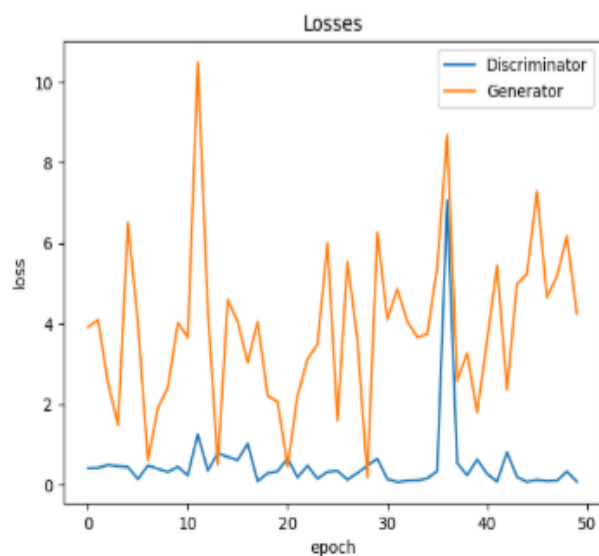
```
Image('./generated/generated-images-0010.png')
```



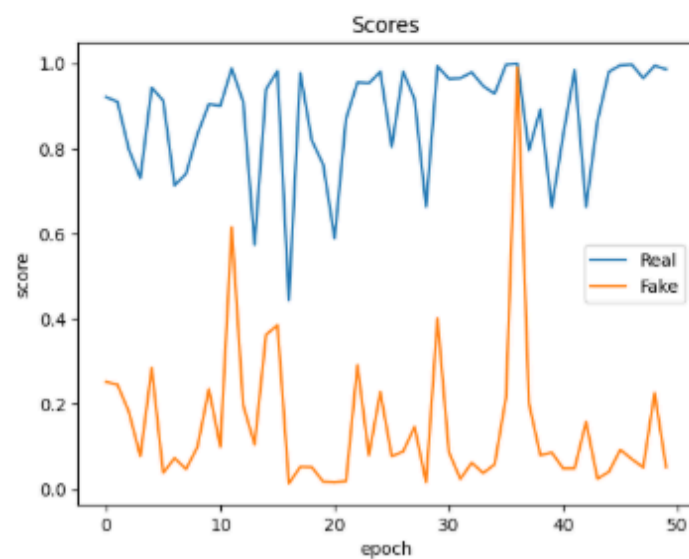
```
Image('./generated/generated-images-0050.png')
```



```
plt.plot(losses_d, '-')
plt.plot(losses_g, '-')
plt.xlabel('epoch')
plt.ylabel('loss')
plt.legend(['Discriminator', 'Generator'])
plt.title('Losses');
```



```
plt.plot(real_scores, '-')
plt.plot(fake_scores, '-')
plt.xlabel('epoch')
plt.ylabel('score')
plt.legend(['Real', 'Fake'])
plt.title('Scores');
```



Experiment-11

Aim : To implement Object detection using YOLO, RCNN, fast CNN, Faster RCNN, other pretrained models in Tensorflow 2.0.

Software Used : Google Colab

Dataset used : Pothole Detection Dataset

Theory :

Object detection involves identifying and localizing multiple objects within an image. In this experiment, we implement various models: YOLO (You Only Look Once) for real-time detection, RCNN, Fast RCNN, and Faster RCNN for region-based detection, along with other pretrained models available in TensorFlow 2.0. These models differ in their speed and accuracy trade-offs. YOLO treats detection as a regression problem, while RCNN variants use region proposals followed by classification. Model performance is evaluated using metrics like mean Average Precision (mAP) and Intersection over Union (IoU).

Source code :

```
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
import cv2
import matplotlib.pyplot as plt

MODEL_URL = "https://tfhub.dev/tensorflow/ssd_mobilenet_v2/2"

detector = hub.load(MODEL_URL)

# Load Labels
LABELS_URL = "https://storage.googleapis.com/download.tensorflow.org/data/mscoco_label_map.pbtxt"
labels_path = tf.keras.utils.get_file(fname="mscoco_label_map.pbtxt", origin=LABELS_URL)

def load_labels(path):
    categories = {}
    with open(path, 'r') as file:
        lines = file.readlines()
        for line in lines:
            if "id:" in line:
                idx = int(line.split(":")[-1])
            if "display_name:" in line:
                name = line.split(":")[-1].strip().strip('\"')
                categories[idx] = name
    return categories
```

```
# Bounding box coordinates
h, w, _ = img_copy.shape
ymin, xmin, ymax, xmax = bbox
left, right, top, bottom = int(xmin * w), int(xmax * w), int(ymin * h), int(ymax * h)

# Draw box
cv2.rectangle(img_copy, (left, top), (right, bottom), (0, 255, 0), 2)

# Label
label = labels.get(class_id, "Unknown")
cv2.putText(img_copy, f"[label]: {score:.2f}", (left, top - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)

plt.figure(figsize=(10, 8))
plt.imshow(img_copy)
plt.axis('off')
plt.show()

visualize(img_resized, result)
```

```

# Bounding box coordinates
h, w, _ = img_copy.shape
ymin, xmin, ymax, xmax = bbox
left, right, top, bottom = int(xmin * w), int(xmax * w), int(ymin * h), int(ymax * h)

# Draw box
cv2.rectangle(img_copy, (left, top), (right, bottom), (0, 255, 0), 2)

# Label
label = labels.get(class_id, "Unknown")
cv2.putText(img_copy, f"{label}: {score:.2f}", (left, top - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)

plt.figure(figsize=(10, 8))
plt.imshow(img_copy)
plt.axis('off')
plt.show()

visualize(img_resized, result)

```

Output :

